OxyMake: A Content-Addressed Workflow Engine

Emmanuel Sérié
Centre de Mathématiques Appliquées (CMAP), CNRS,
École Polytechnique, Institut Polytechnique de Paris,
91128 Palaiseau Cedex, France

August 16, 2026

Abstract

A workflow engine's job is to decide which results survive a change to code, data, or machine. Almost every engine answers with information local to where the work happened—a file timestamp, or a provenance record written beside one copy of the project—so the answer is worthless anywhere else: the next machine, the next collaborator, the next checkout recomputes what is already correct. Engines that decide by content instead usually buy the property with infrastructure: a background service, or a store outside the project.

OxyMake is built on the position that the validity of a result should be a property of the work, not of the place it ran, and that this need not cost an operational system. A result's identity is a pure function of what its rule declares: command text, inputs, parameters, environment. On a given platform and version of the tool, the same declaration yields the same identity in every copy of a project, decided by a single Rust binary with nothing running behind it. Identity is not reuse—results must still travel—but it is what makes reuse decidable at all.

We describe the design, measure repeated execution, and model-check three specifications of the state layer over a bounded space of states. The guarantees reach exactly as far as the workflow declares, and no further.

1 Introduction

The validity of a result should be a property of the work, not of the place it ran. Most engines decide otherwise. Asked which declared outputs still have a valid derivation, they answer from information local to one working tree—a file timestamp, or a provenance record written beside one copy of the project—so the answer does not survive the move to a second machine, a fresh checkout, or a collaborator, each of which starts with nothing and recomputes what is already correct. OxyMake[1] takes the opposite position, and takes it with no operational system behind it. `ox run` ensures the declared outputs exist, executing only the jobs whose outputs are missing or out of date. The operation is convergent: repeating it performs only the work that remains, and on an up-to-date tree it does nothing. Five commands share this model and a common filter surface—`ox run` (converge), `ox cancel` (abort), `ox invalidate` (reset), `ox plan` (preview), and `ox status` (observe) (§4.4).

[1] Source code and project site: `https://github.com/noogram/oxymake` (`https://oxymake.noogram.dev`). Repository paths cited in this paper refer to tag `v0.1.0`.

Single-binary execution of a declarative file. Requiring no operational system is a design constraint rather than a packaging property: it is what lets the key be recomputed in a checkout nobody provisioned. The workflow is a declarative TOML file; a three-line rule with `input`, `output`, and `shell` is a valid workflow. It is executed by a single 14.9 MB Rust binary that bundles no interpreter and needs no daemon; `ox -help` returns in 6 ms (§6.6, §6.7). The same file runs sequentially, in parallel (`-j N`), or—by changing one flag—on a Slurm cluster or under Ray (§2.4). The evaluation in this paper is single-machine; no distributed performance is measured (§6).

A machine-readable command interface. Every command accepts `-json`; when it is given, output is newline-delimited JSON (NDJSON). Gate approval is a command rather than a terminal prompt, so a script or an AI agent drives the same operations a human does (§5.4). An MCP server (`ox serve -mcp`) exposes a read-only inspection subset of the engine over the Model Context Protocol. This is an agent-compatible interface; we do not measure agent task success and make no claim that OxyMake improves it.

A content-derived rebuild decision. Each job's cache key (Eq. 1) is a BLAKE3 hash of the rule source, the declared input contents, the parameters, the environment specification, the shell, and the platform; it records no state private to the working tree the job ran in. When that declared specification is unchanged and its inputs lie inside the workflow root, the *computation key* is therefore identical in any checkout on the same platform and under the same key-format version (v in Eq. 1). Snakemake 7 instead records rebuild provenance per working tree [17, 24]; bound to one tree, that record does not travel, so a fresh clone starts without it even when equivalent outputs have been restored from elsewhere—a consequence of the record's scope, not a scenario we benchmark. What a shared key buys is decidability rather than reuse. Whether two checkouts take the same *rebuild decision* depends further on their live reuse state—the local key-to-output index, the outputs materialised on disk, and the selected validation policy—which §3.1 treats, along with what does and does not travel between checkouts in v0.1.0. What the shipped system establishes across checkouts is key identity.

Scope of the content-key claim. The key is sound over declared inputs only. OxyMake does not sandbox rule execution, so an undeclared read—a helper script, a tool on `$PATH`, an environment variable outside the declared specification—never enters the key and can produce a stale reuse (§3.1). Output verification depth is a policy: the default `mtime+hash` re-hashes a file when its size or timestamp changes, `mtime` trusts metadata for parity with existing runners, and `hash` re-reads every byte (§3.1). Timestamps serve here as an experimental control: our benchmark bumps a shared input's mtime without changing a byte—what a `git checkout` does to a working tree—and counts re-runs. Two of the three outcomes are measured: Snakemake 7.32.4 re-runs zero jobs, because its recorded provenance ignores live timestamps, and a pure-mtime policy re-runs the full downstream radius. The third is true by construction and the benchmark only confirms it: under the `mtime+hash` default a changed timestamp triggers a re-hash, the re-hash finds the same bytes, and no job can be scheduled—zero re-runs (§6.3). The phantom re-run is real for GNU Make [13] and for any engine's pure-mtime fast path, and is not a difference from the benchmarked Snakemake.

Results. On a synthetic 10,001-job workflow on one machine, OxyMake resolves the job graph in 69 ms against Snakemake 7.32.4's 2.31 s, a 33.3× difference on graph resolution (§6.1). A cold end-to-end run is 1.25–2.3× slower, because the content-key bookkeeping is written on

the cold path; a warm no-op re-run is 4.02–7.54× faster, depending on validation policy (§6.2). Because concurrent `ox run` processes create more interleavings than a test suite can sample, three TLA+ specifications [19] model-check bounded safety properties of the concurrent state protocol (§3.5); this is the hazard class for which Amazon Web Services adopted TLA+ at production scale [26, pp. 66–73].

Why a new engine. No single property above is new. The combination is new among the systems we compare (Table 14, §2); we are not aware of a system outside that set that combines them, but we have not run a systematic survey. We chose to build the combination in rather than retrofit it. In the file-rule engines and build systems closest to OxyMake's design point—GNU Make, Snakemake, Bazel, Buck2, and Nix—the default rebuild decision is entangled with a state model or a deployment premise that an adaptation would have to move: GNU Make's default decision reads live mtimes, per-tree state by construction; Snakemake's default decision reads a per-tree provenance record, and its opt-in between-workflow cache (`--cache`) grafts a content-derived key beside that record as a per-rule side channel rather than replacing the default (§2.1); Bazel, Buck2, and Nix take the daemon or the store as an operating premise. (DVC is the acknowledged exception: its default `dvc repro` decision already reads content hashes with no daemon, over pipeline stages rather than wildcard file rules—§2.1.) Making a key that is a pure function of declared content—holding no per-tree state and requiring no external infrastructure—the *default identity for cached computations* would in each case mean moving that boundary. That identity is not by itself the rebuild decision: as above, the decision also consults the local key-to-output index, the outputs materialised on disk, and the selected validation policy. We have not attempted those adaptations; building the engine around the key is a design choice, not an impossibility claim.

Contributions. This paper contributes:

1. **Convergent, daemon-free execution.** `ox run` reconciles declared outputs against the current tree and is safe to repeat, under the stated assumption that declared rules are safe to re-run (§4.4). A persistent claim-state protocol is implemented in the state layer and model-checked; it is not yet the scheduling gate, so overlapping sessions currently re-execute shared jobs independently (§3.5).

2. **Single-binary implementation with pluggable executor backends.** A 14.9 MB Rust binary with a backward-chaining resolver runs the same workflow from a laptop to Slurm or Ray without editing the workflow (§5, §2.4).

3. **A machine-readable execution interface.** Structured NDJSON events and command-based gate approval let a program or agent consume and drive a run without scraping terminal output; an MCP server exposes a read-only inspection subset of these operations (§5.4).

4. **Content-addressed caching by default.** The cache key is a BLAKE3 hash of rule source, declared input contents, parameters, environment, shell, and platform (Eq. 1); the key is identical when that entire declared specification is identical. Output content is checked at reuse time by a configurable validation policy against the local key-to-output index, and reuse cascades on deletion (§3.1).

5. **Three-graph architecture** (RuleGraph → JobGraph → ExecGraph) separating rule definition, resolved-and-pruned plan, and per-run execution trace [14] (§4.1).

6. **A plan of record (`ox.lock`)**: a content-hashed lockfile that re-derives the same job graph from the same TOML, on the same platform, under the recorded assumptions (§3.1, §6.9).

7. **Three TLA+ specifications** totalling 623 lines that model-check bounded safety of the concurrent state protocol (§3.5).

Running example. To ground the discussion that follows, Listing 1 shows a complete Oxymakefile for a three-stage data pipeline. The file is valid TOML; no embedded Python or custom DSL is needed. Key concepts introduced here—wildcards, named inputs/outputs, `expand`, and the `all` pseudo-rule—are defined precisely in Sections 3–4.

Listing 1: A complete Oxymakefile: generate per-sample data, compute statistics, and merge into a report.

```
ox_version = "0.1"

[config]
samples = ["alpha", "beta", "gamma"]

[rule.all]
input = ["results/report.txt"]

[rule.generate]
output = ["data/{sample}.csv"]
shell = """
echo "word,count" > {output}
for w in the {sample} pipeline workflow; do
    echo "$w,$(( $(printf '%s' "{sample}-$w" \
        | cksum | cut -d' ' -f1) % 100 + 1 ))" >> {output}
done
"""

[rule.stats]
input  = { csv = "data/{sample}.csv" }
output = { txt = "results/{sample}_stats.txt" }
shell  = """
echo "# {sample}: $(tail -n+2 {input.csv} | wc -l) rows" \
     > {output.txt}
"""

[rule.report]
input   = ["results/{sample}_stats.txt"]
output  = ["results/report.txt"]
expand  = "product"
shell   = """
echo "=== Pipeline Report ===" > {output}
cat {input} >> {output}
"""
```

The resolver reads the `all` rule, traces its input `results/report.txt` back through `report` → `stats` → `generate`, and expands the `{sample}` wildcard against the three configured values: three `generate` jobs, three `stats` jobs, and one `report` job whose input

list expands (`expand = "product"`) to the three per-sample stats files—a seven-job DAG (`all` is a pseudo-target, not a job). The word counts are derived from a checksum of the sample name and word, so re-running the workflow reproduces the same bytes. Content-addressable caching ensures that a re-run after editing only `rule.stats` skips the three `generate` jobs entirely. We refer to this example throughout the paper.

2 Background and Related Work

2.1 File-Based Workflow Systems

The lineage of file-based workflow systems begins with Make [13], which introduced the fundamental abstraction: rules declare outputs, inputs, and commands; the engine constructs a directed acyclic graph (DAG) and executes only the steps whose outputs are missing or out of date. Every system below inherits some part of Make's model, but the two halves have different fates: the backward-chaining resolution model is widely adopted, while the mtime freshness test that Make pairs it with is not. Several of the systems below adopt the former and reject the latter, and at least one—CWL—adopts neither.

Snakemake [17] extended Make with Python-based rule definitions and multi-wildcard pattern matching, making it the most widely used workflow system in bioinformatics. Its 2021 update added containerized execution, cloud integration, and module composition [24]. Snakemake rules embed Python, so the full job graph is known only after the workflow file is executed; static analysis and IDE support are correspondingly less complete than for an inert format, not absent. In the 7.x line its change detection adds a recorded per-output checksum to the live mtime comparison rather than replacing it: an input counts as updated only if it is newer than the oldest output *and* its recorded SHA-256 no longer matches, and a checksum is recorded only for inputs smaller than 100,000 bytes. Below that threshold the checksum decides, and the decision survives `git checkout` mtime churn unmoved (verified at every bench scale, §6.3); above it the mtime comparison alone decides. The record is bound to a single working tree and does not validate output content, so it is not valid on another machine and does not detect on-disk corruption. Snakemake also ships an opt-in between-workflow cache (`--cache`, with per-rule `cache: True`) whose entry name is derived from rule, input, and software-environment content; it is a per-rule side channel next to the provenance record, and does not replace the default rebuild decision.

A neighbouring family versions data pipelines rather than executing file rules: DVC [18] records content hashes of stage inputs and outputs in `dvc.lock`, a file that travels with the repository, and takes the `dvc repro` rebuild decision from those hashes with no daemon and an in-project cache. It is a Python runtime layered on Git rather than a wildcard-driven file-rule engine, and its machine interface does not extend to a structured event stream, so it does not occupy the combination claimed here (§7)—but it shows that a content-hash rebuild decision without a daemon predates this work.

Nextflow [11] takes a dataflow-oriented approach with channels connecting processes, with strong containerization support. Its `-resume` rebuild decision is an automatic per-task hash over the task script, the container, Conda or Spack specification, the task inputs and the variables the script references, computed with no daemon; input files enter that hash by name, size and last-modified timestamp by default, and by content under the `cache 'deep'` directive. The axis that separates it from OxyMake is therefore the Groovy DSL and the JVM runtime rather than the absence of a content key.

The Common Workflow Language (CWL) [9] occupies a different position in this list: it

is a vendor-neutral standard for describing workflows of command-line tools rather than an engine. Steps are wired by explicit data connections rather than by filename convention, and no change-detection policy is prescribed at all. Its documents declare inbound data links explicitly rather than inferring them. Because the standard fixes what a step consumes and produces without prescribing how it is scheduled, staged, or cached, a conforming engine may optimise freely underneath a workflow its author never edits. `cwltool`, the reference implementation, is scoped for feature-completeness and validation rather than for portability to every host operating system, and already offers an optional cache: with `--cachedir` it fingerprints the realised command, content checksums where available (otherwise size and mtime), container selection, and selected requirements, naming the cache entry from a digest of that dictionary. It is therefore content-sensitive where checksums exist, and metadata-sensitive where they do not; platforms such as Arvados [4] exercise the standard's latitude further, pairing CWL with content-addressed data management. CWL is therefore better read as a complementary layer to this work than as a competitor to it—a description language OxyMake could consume, rather than a change-detection policy it must correct. Galaxy [2] leads with a web-based interface built for biologists; it also exposes a REST API and the BioBlend client library for programmatic use, while Planemo provides a command-line tool for developing and testing Galaxy tools and workflows—rather than the text-first authoring loop this paper assumes. The Workflow Description Language (WDL) [37] and its reference engine Cromwell target genomics pipelines with a typed, portable specification; Cromwell is backend-agnostic, with a pre-enabled Local backend as its default and pluggable HPC, TES and cloud-batch backends behind the same interface. One-shot `cromwell run` executes a workflow as a CLI process and exits; a server deployment is required only for its server-mode operations. Cromwell also offers a call cache, disabled by default, keyed on the task command, the task inputs (input files hashed by content, `md5` by default on a local filesystem) and the `docker`, `continueOnReturnCode` and `failOnStderr` runtime attributes, with a floating image tag resolved to its immutable digest; that reuse is bound to a persistent Cromwell database rather than to the workflow declaration. The cost relative to OxyMake is therefore a separate execution engine, a more verbose specification (§6.5), and a reuse scope tied to a database deployment. Pegasus [10] targets large-scale distributed workflows but requires significant infrastructure.

OxyMake inherits Snakemake's core paradigm—backward-chaining DAG resolution with wildcard-driven genericity—and makes different trade-offs on three axes: a content-addressed cache key in place of per-tree provenance, a declarative TOML format in place of an embedded Python DSL, and a native Rust binary in place of a Python runtime. Each trade-off has a cost, stated where it is measured (§6).

2.2 Build System Theory

Mokhov et al. [22, §3–4] [23, §3] provide the standard theoretical framework for build systems, decomposing them along two orthogonal axes: the *scheduler*, which decides the order in which tasks run (topological: fixed order from the dependency graph; restarting: re-queues a task when a new dependency is discovered; or suspending: pauses a running task mid-flight to resolve a newly discovered dependency), and the *rebuilder*, which decides whether a task's existing output can be reused (dirty bit: rebuild if any input changed; verifying traces: rebuild only if recorded input/output hashes no longer match; constructive traces: select a previously built output whose inputs match; or deep constructive traces: allow intermediate inputs to differ as long as final content matches). Their key insight is that every build system is a composition of a scheduler and a rebuilder, yielding a two-dimensional design space where existing systems

(Make, Shake [21], Bazel [16], Nix) occupy specific cells.

In this taxonomy, OxyMake combines a **topological scheduler** with a **verifying-traces rebuilder** augmented with content-addressing. All dependencies—including wildcard expansions and scatter/gather patterns—are resolved statically by a backward-chaining resolver before the scheduler begins execution. The scheduler then computes a topological order over the fully materialised DAG and dispatches jobs whose upstream dependencies are satisfied, with no runtime dependency discovery or suspend/resume. On the scheduler axis this places OxyMake with Make, Ninja and Buck; the topological/verifying-traces cell it claims is the one Mokhov et al. assign to Ninja. Bazel [16] is not in that column: it is classified as restarting, because it supports action-time input discovery (*discovered inputs*), so its evaluation graph is not fixed before execution in the way an OxyMake JobGraph is. The affinity with Bazel is elsewhere—its analysis phase likewise constructs an action graph before execution, and it likewise caches on content. OxyMake replaces Bazel's Starlark [5] configuration layer with declarative TOML and adds domain-specific features (wildcard expansion, environment management, gates) for scientific workflows. Meta's Buck2 [20] pushes this design further with a fully content-addressed execution model built on the Starlark configuration language; OxyMake shares the content-addressing principle but replaces the Starlark layer with TOML to preserve static parseability.

The distinction between *Applicative* tasks (statically known dependencies) and *Monadic* tasks (dependencies discovered at runtime) clarifies OxyMake's position: its rules are strictly Applicative. Wildcard patterns and scatter/gather configurations *appear* dynamic but are expanded at resolution time—before the job graph is constructed—so the scheduler never needs to suspend a running task to discover new dependencies.

Dolstra's Nix thesis [12, ch. 5] introduced the input-addressed store for software deployment, where store paths encode cryptographic hashes of all build inputs; ch. 6's intensional model extends content-addressing to derivation outputs themselves. OxyMake adopts the input-addressed principle for workflow outputs: the cache key is a BLAKE3 hash of the rule source, input content hashes, parameters, environment specification, and platform. The key is input-addressed; the artefact store beneath it is content-addressed, blobs being named by the BLAKE3 hash of their own bytes. One distinction must be drawn: Dolstra's observation that "if a build succeeds, we know that we have specified all the dependencies" rests on Nix's build *sandbox*, which hides undeclared inputs from the builder so that an incomplete declaration fails loudly. OxyMake adopts the content-addressed key but not the sandbox: rules run as ordinary processes, so the key is comprehensive over *declared* inputs only, and input completeness remains the user's obligation (§3.1).

2.3 Reproducibility and FAIR Workflows

This subsection maps the reproducibility property OxyMake delivers onto the existing FAIR workflow literature, and marks the boundary between what the orchestration layer records and what it delegates to the binary substrate.

The Goble three-layer model. Goble et al. [14] argue that computational workflows are first-class FAIR (Findable, Accessible, Interoperable, Reusable) digital objects, not merely tools for producing FAIR data. They identify three layers—*abstract workflow* (the rule graph, independent of any instance), *concrete workflow* (the resolved DAG bound to specific inputs), and *execution trace* (the per-run provenance record). Each layer requires independent FAIR compliance. Their observation that "a workflow that cannot be readily reused is like a scientific paper that cannot

be read" is the principle that motivates OxyMake's three-graph architecture (§4.1). The mapping is direct: `RuleGraph` carries the abstract workflow, `JobGraph` (post-resolution, post-cache pruning) carries the concrete workflow, and `ExecGraph` together with the audit-state SQLite tables and the NDJSON event stream carries the execution trace.

The Wilkinson 2025 indicators. Wilkinson et al. [38] operationalise the Goble model with specific indicators for computational workflows. Chue Hong et al. [7] complement this with the FAIR4RS principles for research software, which apply to the OxyMake binary itself as a research artefact. The two papers together define the acceptance criteria a workflow engine must meet to be considered FAIR-native; we report against them in Table 12 and discuss the residuals in §6.9.

The four-layer FAIR reproducibility ladder. Inside the Goble model there is a finer-grained ladder that is useful for placing OxyMake in the existing landscape.

Layer	Witness	Tool examples
L1 – Substrate	Same input hash ⇒ same binary	Guix store, Nix store, Docker digest
L2 – Orchestration	Same plan hash ⇒ same DAG	OxyMake `ox.lock`, Snakemake `report`
L3 – Execution	Same DAG ⇒ same output hashes*	OxyMake content-cache, Bazel actions
L4 – Audit	Run trace decoupled from code	OxyMake NDJSON + `state.db`, RO-Crate

Table 1: A four-layer FAIR reproducibility ladder. Each layer provides an independent witness, and the witnesses *compose* rather than overlap. A workflow runner can provide the L2–L4 witnesses *without* providing L1, provided it does not report the substrate's guarantees as its own. *The L3 witness holds only for deterministic rules on a fixed substrate.

Delegating the substrate layer. OxyMake's cache key includes an *environment specification* (e.g., `requirements.txt` content hash, Docker image reference, Guix manifest hash), but treats the substrate's contract as an out-of-model axiom (§3.5, substrate boundary). The engine does not attempt to verify that a `uv.lock` or a Guix manifest is itself bit-reproducible; it records the hash and trusts the substrate to deliver. This delegation is deliberate: it lets a FAIR-archival reader reproduce the OxyMake-level witnesses (L2–L4) on any substrate that honours the recorded hashes. The guix-cwl reference workflows [31, 39] *illustrate* one such substrate-composition pattern—a CWL workflow run under a Guix-managed environment, with Guix providing L1 and the workflow runner providing L2–L4. Whether OxyMake's orchestration-level contract composes cleanly with such a substrate is a design conjecture and a future direction (§7.4), not a property attested here.

The reproducibility crisis in computational science [29, 34] underscores the need for workflow systems that guarantee deterministic re-execution—and equally underscores the need for explicit scope-marking: a runner that reports the substrate's guarantees as its own delivers a false signal when the substrate changes. OxyMake's content-addressable caching provides a stronger guarantee than timestamp-based systems: same inputs + same rule + same parameters ⇒ same cache *key*, on any machine of the same OS/architecture with the same layout inside the workflow root. The cache key is content-derived, whereas verification of an existing artefact depends on the selected policy (§3.1). And the guarantee holds *only at L2–L4*. L1 is delegated, by design, to the substrate of the reader's choice.

2.4 Distributed Execution

DAG-based workflow scheduling on heterogeneous distributed systems is a well-studied problem. The HEFT (Heterogeneous Earliest Finish Time) algorithm [35] and its variants provide efficient heuristics for task placement. Adhikari et al. [1] survey cloud scheduling strategies, identifying the trade-off between makespan minimization and cost optimization.

Apache Airflow [6] popularised DAG-based workflow orchestration for data engineering, modelling pipelines as Python-defined task graphs with pluggable executors. Argo Workflows [3] applies the same DAG model natively on Kubernetes, encoding steps as container specifications in YAML. Both systems excel at scheduling heterogeneous tasks across distributed infrastructure. Neither, however, *automatically* derives a reusable result key from the content of all declared file inputs: Airflow's graph is built by executing Python, so its structure is available only after the DAG file has been run, and Argo's YAML—declarative in form—names container images and command lines without hashing the data flowing between steps. Argo does expose a memoization mechanism (`memoize:`, since v2.10) whose key is user-supplied, so a user may construct a content-derived key by hand; there the contrast is the absence of an automatic declared-file-content key, not an impossibility. Airflow exposes no task-result memoization primitive at all: a scheduled task instance is executed.

Frameworks like Ray [25] and Dask [32] provide distributed execution with task-level parallelism, but their principal surface is a programming API rather than a declarative file-rule model: Dask is Python-native, and Ray is multi-language (Python, Java, and a C++ API) while still exposing tasks and actors rather than file rules. OxyMake bridges this gap with its executor backends: the same declarative workflow runs on a local machine (`-j N`), a Slurm cluster (`--executor slurm`), or Ray (`--executor ray`) without modification; a Kubernetes executor is designed but not yet implemented (§6.8).

3 Design Principles

OxyMake's central design contract is convergence:

> *`ox run` reconciles declared outputs against the current tree. It runs the jobs that are missing or out of date and no others, and is safe to repeat.*

Two supporting principles make that contract auditable and transportable, and both are written down with distinct scopes. The first fixes what the engine attests and what it delegates: *the engine records the orchestration-level witnesses (L2–L4) and delegates L1 to the substrate* (§2.3, Table 1). The second fixes the language boundary: *Rust provides the engine, the workflow provides the intent.* This is a separation of concerns: the engine handles DAG resolution, scheduling, caching, and execution mechanics; the workflow definition declares rules, dependencies, and resource requirements. The engine applies no heuristics and no hardcoded thresholds, and the workflow specifies no scheduling logic and no cache management. Each of the principles below supports one or more cells of the reproducibility ladder (Table 1).

Goble three-layer mapping. Each of the six following principles is annotated with the Goble layer(s) [14] it supports: A = abstract workflow, C = concrete workflow, T = execution trace. Content-addressable caching (§3.1) supports C+T; daemon-free execution (§4.4) supports T; the machine-readable interface (§5.4) supports T's machine-readability; the executor backends support A's portability; named invariants and formal specifications (§3.5) support

the trustworthiness of the T-level audit trail; the declarative TOML choice supports A's static parseability.

3.1 Content-Addressable by Default

The cache *key* is derived from file content, not timestamps; whether an unchanged file is re-read or trusted on an unchanged `(mtime,size)` pair is the validation policy described below. The cache key for a job is:

$$k = \text{BLAKE3}\big(v \parallel h_{\text{rule}} \parallel h_{\text{inputs}} \parallel h_{\text{params}} \parallel h_{\text{env}} \parallel h_{\text{shell}} \parallel p\big) \tag{1}$$

where v is a key-format version tag (bumping it cleanly invalidates caches written under an older format), h_{rule} is the hash of the rule's source (command, inline code, script reference, or function reference), h_{inputs} is the sorted sequence of $(\text{path}, \text{content hash})$ pairs covering declared inputs, parameter files, and—in script mode—the script file itself, h_{params} captures parameter values, h_{env} captures the environment specification by *content* (e.g., the bytes of the referenced `requirements.txt` or conda YAML, not just its path), h_{shell} is the configured shell executable, and p encodes the platform (OS + architecture). Every component is length-framed with a domain-separation tag, and optional components carry explicit presence tags, so the pre-hash encoding is unambiguous: two distinct job specifications serialize to two distinct byte streams. The key itself is a 256-bit BLAKE3 digest of that stream, so distinct streams collide with the cryptographic hash's negligible but non-zero probability. Binding each content hash to its path prevents two inputs from exchanging contents without changing the key. Because p is part of the key, cache entries are shared only between machines of the *same* platform; heterogeneous OS/arch cache reuse (e.g., develop on macOS arm64, run on Linux x86_64) never produces a false hit—and never hits at all—and is left as future work. Each input's *path* is hashed together with its content, which binds a hash to the file it was computed for but also makes path spelling part of the key. Paths are therefore recorded *relative to the workflow root*—defined as the directory `ox` is invoked from, not the parent of the file passed via `-f`. Before entering the key, a relative path is interpreted from that root, `.` and `..` components are resolved lexically, and paths that exist on disk are additionally canonicalised so that a symlink pointing outside the root is seen for what it is. A path that lands *inside* the root after this normalisation is recorded relative; one that escapes it—via an absolute spelling, a `..` prefix, or a symlink—is recorded as a normalised absolute path, because its location is part of the workflow's dependency and is machine-specific. Two checkouts of the same tree at different absolute locations thus produce identical keys for in-root inputs, which makes the key independent of the root's absolute location and therefore identical across machines. Key identity requires the same OS/architecture and the same layout *within* the workflow root, not the same location of that root. Two residual exclusions remain by design and are documented in §7.3: `call`-mode function bodies (the referenced module is content-tracked only if declared as an input) and mutable container image tags (hashed as written, not resolved to digests).

Cache validation is **pluggable**: users choose one of three strategies through the `--cache-validation` option:

- `mtime+hash` (default) — an unchanged `(mtime,size)` pair is trusted; if mtime or size differ, compute the BLAKE3 hash before declaring a hit or miss. Fast on steady-state, correct on change: same-size corruption with a newer timestamp is detected rather than served. The guarantee is bounded by what it watches—a rewrite that preserves both size and mtime never triggers the hash and is served as a hit, and this applies both to a

previously stored input hash and to an existing output. This policy addresses corruption that changes size or timestamp; `hash` below also covers rewrites that preserve both.

- `mtime` (opt-in) — pure filesystem metadata (stat calls only). Matches Make-style metadata-only behavior; delivers sub-100 ms no-op runs with no dependency on `.oxymake/`. Content is never verified, so this mode is unsuitable for shared or multi-user caches.

- `hash` — always compute BLAKE3 hashes, unconditionally re-reading every byte. The policy to use for CI reproducibility audits, and the one a shared or remote cache would require.

The strategy is configurable per invocation (`–cache-validation`), per project (`[config] cache_validation` in Oxymakefile.toml), per environment (`OX_CACHE_VALIDATION`), or globally (`~/.config/oxymake/config.toml`). When a remote cache is in play (`ox run –cache-remote <dir>`, the shipped directory backend), the validation strategy is promoted to `hash` regardless of what was configured: a shared store has no meaningful mtime relationship with the local workspace, so every restored artefact is verified by content. The evaluated remote-cache implementation is a shared-directory blob store: it stores content-addressed artefact bytes, while the SQLite index that maps a computation key to its output paths and hashes remains local to each checkout, so restoring into a fresh checkout requires copying that index separately. A computation-key-addressed manifest stored remotely, which would make the directory backend self-sufficient, is future work; S3 and GCS backends are not implemented (§6.8).

All dimensions of the cache key must be present from the initial release. Adding a missing dimension later would invalidate the entire cache for all users—an unacceptable cost.

Threat model: undeclared inputs. Equation 1 is sound exactly over what it hashes: *declared* inputs. OxyMake executes rules as ordinary processes, with no sandbox or syscall-level isolation. Anything a rule reads without declaring it—a helper script invoked by the shell command, a binary on `$PATH`, a configuration file, a locale setting, an environment variable outside the declared environment specification—never enters the key, so a change to it produces a *false cache hit*: the engine reuses a stale output while reporting that nothing changed. This is the inverse of the phantom re-run, and it is the more dangerous failure because it is silent. Nix and Guix close this hole with a build sandbox that makes undeclared reads fail at build time; OxyMake deliberately does not (rules must run unmodified user code on hosts where namespace isolation is unavailable or unwanted), so the completeness of the input declaration is trusted, not enforced. The practical mitigations are discipline (declare scripts and tools as inputs; pin the environment via `uv.lock` or an image digest, which folds it into h_{env}) and audit (the `ox.lock` plan of record makes the declared key inputs inspectable). Sandboxed execution as an opt-in enforcement layer is future work (§7.3). Consequently, cache correctness assumes complete input declarations.

3.2 Daemon-Free Execution

No daemon process is required. Each `ox run` invocation is self-contained: it resolves the DAG, executes jobs, writes state to `.oxymake/state.db` (SQLite), and exits. The state layer implements a persistent claim-state protocol of optimistic-lock SQL transitions specified by `CooperativeClaim.tla` in TLA+ [19]: its invariant **INV-2** states that no two sessions hold

the same job and that a stale session's lease is reclaimed (§3.5). This protocol is implemented and model-checked but is not yet used as the scheduling gate. In consequence, two `ox run` processes whose job sets overlap currently execute the shared jobs independently rather than one claiming and the other waiting; this is safe under the paper's standing assumption that a declared rule is safe to re-run, and state writes are atomic. No long-running control-plane service is required. An optional `ox serve` mode is available for long-running orchestration but is never required for a local run.

3.3 Command and Library Interfaces

Every operation is available as a CLI command with `--json` output, so the same invocation serves a human and a program. The engine's operations also live in library crates callable from Rust; a single-facade crate (`ox-api`) exposes them through a typed entry point (`SessionBuilder`, `plan`, `run`). The facade is pre-1.0 and its surface may change; the command surface is the interface this paper evaluates:

```
# Human-readable
ox run results/all.vcf.gz -j 8

# Machine-readable (same operation)
ox run results/all.vcf.gz -j 8 --json
```

3.4 Executor Backends

The same workflow file runs at every scale; moving from a laptop to a cluster changes one flag, not the workflow. Three executor backends are implemented—local, Slurm, and Ray—where the local backend runs sequentially by default and in parallel with `-j N`; the Kubernetes executor is designed but not yet implemented (§6.8):

Level	Flag	Mechanism
Sequential	(default)	Single process
Local par.	`-j N`	Tokio thread pool
Slurm	`--executor slurm`	`sbatch`/`sacct`
Ray	`--executor ray`	Ray Jobs API
Kubernetes	`--executor k8s`	K8s Job via kube-rs *(not yet implemented)*

All entries require the `.oxymake/` state directory on local disk. It is created in the process working directory and cannot be relocated, so in practice the requirement is on the *workspace*: the directory `ox run` is invoked from must be on local disk, together with the inputs and outputs resolved relative to it. On a cluster the scheduler runs on the submission node and holds the SQLite state there; compute nodes run jobs and never touch the database. Multi-node coordination across NFS, Lustre, or GPFS, and coordination across multiple submission nodes, are future work (§7.3). The evaluation is single-machine; no Slurm or Ray run is measured (§6).

3.5 Named Invariants and Formal Specifications

We use bounded model checking to examine safety properties of concurrent state transitions; this discipline backs the L4 audit-trail witness of the FAIR ladder (Table 1).

OxyMake's runtime is built from components that are individually deterministic—a worker, a scheduler, an evictor, a cancel path, a state database—yet whose composition is concurrent. A FAIR-archival reader has no way to distinguish "the workflow reproduced" from "the workflow ran once on a single-session, no-fault path that happened to produce the same bytes." The discipline that lets us *claim* reproducibility on the multi-session, fault-tolerant path is formal specification of the cross-session safety properties. Without it, the L4 audit trail is a log of one history out of an unknown number of possible histories.

Concurrent sessions may fail independently while updating shared state; we therefore model claim, cancellation, and recovery transitions jointly. On at least three surfaces (multi-session reclaim, cancel propagation, and post-crash recovery) the state of the system is a *relation* between the states of peers that can each fail independently. The reachable interleavings on those surfaces far outnumber what any feasible test suite can sample; a green CI corroborates a very small fraction of the state space.

The structural reference is Newcombe et al. [26, pp. 66–73], which reports seven bugs found by TLC in ten AWS systems—bugs that had been missed by testing, code review, static analysis, stress testing, and fault injection. Those systems were orchestrated on deterministic components; the hazard class was concurrent, not adversarial. The structural similarity to `ox run` is what justifies the technique here, not any claim about input non-determinism.

Scope and ratio. OxyMake ships three TLA+ specifications totalling 623 lines (`.tla` source: `CacheConsistency.tla` 201 L, `CooperativeClaim.tla` 254 L, `CancelPropagation.tla` 168 L). What these specifications verify, and at what bounds, is stated in Table 3 and Appendix A; that scope, not any line count, is the measure of the effort. Against 64,596 SLOC of Rust workspace code the specifications are 0.96% of the source, the same order of magnitude as the ∼1% Newcombe et al. [26] report; the figure is descriptive context, not a claim of comparable coverage. Post-crash recovery is outside the evaluated scope.

Mapping invariants to specifications. Each specification checks one or more named invariants, derived from the properties the implementation depends on:

Invariant	Property	Spec
OX-1	cache key determinism	`CacheConsistency.tla`
OX-2	no backchannel (info direction)	— (architectural, not model-checked)
OX-6	stationary cache safety	`CacheConsistency.tla`
INV-3a	`CancelledNeverCached`	`CancelPropagation.tla`
INV-2	claim atomicity / stale-session reclaim	`CooperativeClaim.tla`
INV-3b	`JobFailedImpliesNoIntent`	`CancelPropagation.tla`
INV-3c	`NoZombieRunning`	`CancelPropagation.tla`

Table 2: Named invariants and the specifications that check them. OX-1, OX-2, and OX-6 name product-level properties; INV-2 and INV-3 name the concurrent safety properties checked by TLC.

Bounds. "Model-checked" here is a claim about *bounded model checking*, not proof. TLC exhaustively explores the reachable state space of each spec at small, fixed instance sizes: `CacheConsistency` at 2 workers × 3 rules, `CooperativeClaim` at 3 sessions × 2 jobs (TTL 2, clock bound 4), `CancelPropagation` at 3 jobs. Within those bounds the invariants hold

over *every* interleaving—this is the qualitative step beyond testing, which samples interleavings. Beyond them the invariants are corroborated, not verified; no inductive proof (TLAPS) has been attempted.

Assumptions. The specs import axioms they do not check (next paragraph); the model-checked guarantee is conditional on those axioms holding in deployment. In addition, OX-1 as model-checked is conditional on key purity, and that condition is itself modelled: in `CacheConsistency.tla` each materialisation draws the rule's key from a constant set `KeyVariants`, and a worker crash before registration lets a peer recompute the key for the same rule. Under the shipped configuration (`KeyVariants = {1}` — the key is a pure function of the rule's declared inputs) `CacheKeyDeterminism` holds over all crash/re-claim interleavings; a second configuration (`KeyVariants = {1,2}`, modelling an undeclared input leaking into the key) refutes it, which is the evidence the invariant has content rather than being true by construction. Purity of the real BLAKE3 key remains an assumed property, and the input-declaration completeness assumption (§3.1) remains out of model.

Substrate boundary. The specifications import named axioms about SQLite, the filesystem, the kernel, and the executor process; the right column of Table 3 enumerates them. These assumptions are explicit and are not checked by the model. One imported axiom deserves emphasis because it can fail in exactly the deployment where multi-session execution is most likely to be attempted—several sessions pointed at one workspace on a shared filesystem: `CooperativeClaim.tla` assumes **`StateDbAtomicCommit`**—that a SQLite `COMMIT` is all-or-nothing with respect to concurrent writers, so the `reclaim_stale_jobs` transaction is atomic. SQLite delivers that on a local filesystem with working POSIX locks; on NFS, Lustre, or GPFS—shared filesystems where one might naturally point several sessions at one workspace—file locking is unreliable and the premise is *false*. This is why OxyMake requires `.oxymake/` on local disk (§7.3): the requirement is what discharges the axiom. It is a requirement on the operator, not a mechanism the engine provides—`.oxymake/` is created in the working directory and there is no way to place it elsewhere, so satisfying it means putting the whole workspace on local disk. If the state database is run on NFS, the premise fails and INV-2's model-checked guarantee no longer applies.

Verified vs. assumed. Table 3 draws the line in one place.

The full specification inventory, with the bounds explored and the state counts, is given in Appendix A.

3.6 Declarative Workflow, Polyglot Execution

The workflow definition is always TOML—declarative, statically parseable, not Turing-complete. This is a deliberate departure from Snakemake's Python DSL. However, individual rules execute in any language through four execution modes forming a spectrum from opaque to optimizable (Section 4.2).

3.7 Errors as First-Class Design

Every error message traces the full causal chain through the DAG: which job failed, what the root cause was (exit code, OOM, missing input), which upstream rule was responsible, and

Verified (TLC, bounded, exhaustive)	Assumed (imported, not checked)
`DoneByClaimHolder`, `AtMostOneClaimer` (INV-2; 3 sessions, 2 jobs; the zombie-terminal-write red configuration refutes the former when the session filter is disabled)	`StateDbAtomicCommit` — SQLite commit atomicity; holds on local disk, *false on NFS/Lustre/GPFS*; discharged by the local-disk requirement (§7.3)
`CacheKeyDeterminism`, `RegisterPrecedesRead`, `NoDoubleRegister` (OX-1/OX-6; 2 workers, 3 rules, worker crashes; the nondeterministic-key red configuration refutes the first)	`ExecutorHonest`, `UserCodeMatchesRule`, `StorageDeleteAtomic`, `ExecutorFailureClassification` (substrate axioms)
`CancelledNeverCached`, `JobFailedImpliesNoIntent`, `NoZombieRunning` (INV-3; 3 jobs)	BLAKE3 key purity (a pure function of declared inputs — `KeyVariants = {1}` in the model) and input-declaration completeness (§3.1)

Table 3: What the TLA+ suite verifies versus what it assumes. The left column is exhaustively checked by TLC at the stated bounds; the right column is the conditional premise of every left-column guarantee.

what corrective action is available. In JSON mode, the same structure is machine-parseable, enabling downstream tools to programmatically read error chains and take corrective action.

4 Architecture

4.1 The Three Graphs

OxyMake uses three distinct graph representations at different abstraction levels, following the pattern proven by DataFusion (logical → physical plan), Bazel (target → action graph), and Spark (RDD lineage → stages → tasks).

RuleGraph (logical). The workflow as declared by the user. Nodes are `Rule` objects with unresolved wildcards; edges represent input/output pattern dependencies between rules. The RuleGraph is compact and abstract: one `call` node represents *all* variant-call instances. Operations at this level include cycle detection, rule ambiguity detection, and structural validation. The hierarchical visualization command (`ox dag -group-by stage`) operates on this representation.

JobGraph (physical). The RuleGraph expanded (wildcards resolved, conditional guards evaluated) and then optimized through a series of passes. Nodes are `ConcreteJob` objects with fully resolved wildcards; edges are typed as `Produces`, `Consumes`, or `Blocks`. The JobGraph wraps a `petgraph::DiGraph` [30] with typed node variants (`Job`, `Output`, `Gate`).

One optimization pass is implemented for the JobGraph and five are proposed with their trait interface defined but no implementation. Cache pruning is the implemented pass; the five proposed passes are marked below:

1. **Cache pruning**: Mark jobs with up-to-date outputs as `Skipped`, using the content-addressable cache (Equation 1).

2. **Task fusion** *(planned)*: Merge sequential `call`-mode jobs into a single process, eliminating intermediate serialization (analogous to Spark stage fusion).

3. **Materialization elimination** *(planned)*: Remove disk writes between consecutive `call`-mode jobs when the executor supports in-memory passing (analogous to DataFusion pipelining).

4. **Group scheduling** *(planned)*: Bundle parallel jobs for batch submission (e.g., one `sbatch` for N Slurm jobs).

5. **Critical path analysis** *(planned)*: Identify the longest dependency chain and prioritize those jobs for earliest dispatch.

6. **Partition planning** *(planned)*: Assign subgraphs to executors based on resource requirements and executor capabilities.

Each pass implements a `trait OptimizationPass` with a single method, enabling composable, testable transformations:
`optimize(&self, JobGraph) -> Result<(JobGraph, PassResult), Box<dyn Error>>`
The `ox plan` command exposes each optimization stage, analogous to SQL's `EXPLAIN`.

ExecGraph (runtime). The JobGraph annotated with live execution state. Each node carries a status (Pending → Ready → Running → Completed/Failed/Skipped), runtime metrics (wall time, peak memory), and log paths. The scheduler dispatches `Ready` nodes, the reporter emits events, and crash recovery serializes the ExecGraph to SQLite for resumption.

The full pipeline is: `Oxymakefile.toml` → parse → `RuleGraph` → resolve wildcards → `JobGraph (raw)` → optimization passes → `JobGraph (optimized)` → annotate with runtime state → `ExecGraph` → schedule and execute. Figure 1 illustrates this progression using the demo word-frequency pipeline, and Figure 2 shows the RuleGraph as generated by `ox dag -format dot`.

4.2 The Execution Spectrum

OxyMake provides four execution modes forming a spectrum from maximum flexibility to maximum optimizability:

Mode	I/O	Memory	Optimize
`shell`	Files only	No	Opaque
`run`	Files only	No	Limited
`script`	Files only	No	Limited
`call`	Files or mem	Yes	Full

The `call` mode is the most optimizable of the four. A rule declares a function reference (e.g., `pipeline.features:compute_features`) that the user is expected to keep free of side effects; OxyMake manages all I/O outside the function. The engine does not enforce this: the cache key covers the function reference, not the function body, so editing the imported module without declaring it as an input can serve a stale result (§7.3). Shell, `run`, and `script` modes have no such gap, because the script content is hashed into the key.

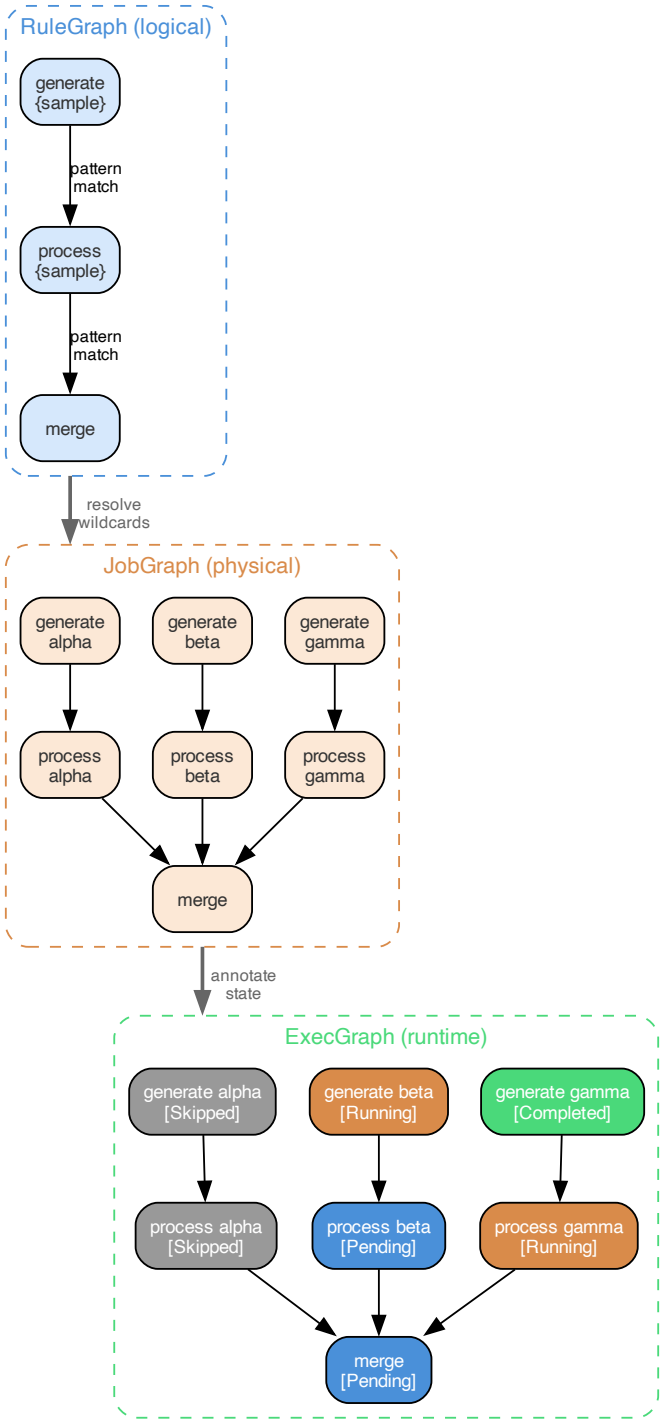

Figure 1: The three-graph architecture applied to the demo pipeline. *RuleGraph* (top): abstract rules with unresolved wildcards. *JobGraph* (middle): wildcards resolved to concrete instances (alpha, beta, gamma), revealing the true parallelism structure. *ExecGraph* (bottom): annotated with runtime state—gray nodes are cache-skipped, orange are running, blue are pending, green are completed.

Figure 2: DAG visualization of the demo word-frequency pipeline generated by `ox dag --format dot`. Rule nodes (blue boxes) are connected through file pattern intermediaries (gray ellipses), forming a bipartite graph that makes data flow explicit. The pipeline progresses left to right: generate → process → merge → target.

```
[rule.compute_features]
input = [{ path = "data/{sample}.parquet",
           format = "parquet" }]
output = [{ path = "features/{sample}.parquet",
            format = "parquet",
            materialize = "auto" }]
call = "pipeline.features:compute_features"
```

The Python function receives a DataFrame, returns a DataFrame, and performs no file I/O:

```
def compute_features(df: pl.DataFrame) -> pl.DataFrame:
    return df.with_columns(
        anomaly=pl.col("value").rolling_mean(20),
        spread=pl.col("value").rolling_std(60),
    )
```

In file mode, OxyMake reads the input using the declared format codec, calls the function, and writes the result. In memory mode, the transfer mechanism depends on the executor: the Ray executor passes data through the Ray object store (`ray.put`/`ray.get`), avoiding disk entirely, while the local executor currently serializes intermediates to disk via format codecs (Arrow IPC transport is planned). The transfer mechanism is invisible to the function.

Materialization policy. The `materialize` field controls when outputs touch disk. Four modes are available: `always` (default, reproducible), `auto` (only if a non-`call` downstream needs the file), `never` (memory only, not cached), and `final` (only DAG leaves). A global override (`ox run --materialize=final`) switches the entire workflow between modes, enabling rapid exploration with reduced disk overhead (zero on executors with native object stores).

Language interop. OxyMake communicates with language runtimes via subprocess and structured serialization rather than embedding (e.g., PyO3). This design choice preserves compatibility with isolated environments (`uv`, `conda`, Docker) and avoids coupling to specific language ABIs. Worker processes are reused across sequential `call`-mode jobs in the same environment to amortize startup cost.

4.3 Plugin Architecture

OxyMake defines five plugin axes, each specified by a Rust trait. Each axis lists the backends implemented today; others named the trait admits but which are not implemented are marked *(planned)*:

Executor Where jobs run: local, Slurm, Ray; Kubernetes *(planned)*.

Storage Where files live: local filesystem; S3, GCS *(planned)*.

Environment How jobs are isolated: system, uv, conda, Docker; Nix *(planned)*.

Reporter How progress is communicated: terminal, NDJSON; webhooks *(planned)*.

FormatCodec How objects are serialized: Parquet, CSV, JSON, and columnar formats.

One placement note reconciles this list with the crate map (§5.3): the shipped environment support (system passthrough, and command wrapping for `uv`, conda, and Docker) is implemented inside the local executor crate (`ox-exec-local`), and local file handling lives in the engine crates. The dedicated `ox-env-*` and `ox-storage-local` crates are extraction targets that today contain trait scaffolding only.

Plugin selection is compile-time via Cargo feature flags. The minimal binary (local executor + local storage + system environment + terminal reporter + basic codecs) has minimal dependencies and compiles fast. Heavy plugins (Kubernetes, S3) are opt-in features. This ensures the default binary remains small and fast to build.

The `Executor` trait is the primary extension point:

```
trait Executor: Send + Sync {
    async fn execute(
        &self, job: &ConcreteJob,
        ws: &Workspace, ctx: &ExecContext
    ) -> Result<JobResult>;
    async fn cancel(&self, id: &JobId)
        -> Result<()>;
    fn capabilities(&self)
        -> ExecutorCapabilities;
}
```

The `capabilities()` method reports whether the executor supports GPU scheduling, streaming between co-scheduled jobs, hermetic workspace isolation, and in-memory passing between `call`-mode jobs. When an executor does not support memory passing (e.g., the local executor's current disk-serialized path, Slurm, Kubernetes), the scheduler automatically promotes `InMemory` outputs to `File`—the workflow runs correctly everywhere, with degraded performance on backends that lack native object stores.

4.4 Idempotent Convergent Execution

`ox run` ensures that the declared outputs exist. This is a declarative, convergent model inspired by `terraform apply`, formalized as a state-machine reconciliation in the sense of Lamport [19]. Every invocation reconciles desired state versus actual state:

Current State	Action
Output cached, inputs unchanged	Skip
Output missing (intermediate deleted)	Re-execute + cascade
Job running (another session)	Re-execute (safe, independent duplication)
Job pending	Execute
Job failed previously	Re-execute

Transitive invalidation of deleted intermediates. The "output missing" row addresses a subtle correctness property that timestamp-based systems miss. When a user deletes an intermediate output file (e.g., `rm results/merged.parquet`), OxyMake's cache pre-scan detects the absence: the cache store verifies that every recorded output *still exists on disk* before declaring a cache hit. A missing file invalidates the producing job's cache entry and triggers **transitive cascade**—all downstream jobs whose inputs transitively depend on the deleted file are marked stale, regardless of whether their own direct inputs appear intact.

This cascade is implemented by propagating staleness through topological traversal of the job graph: if any upstream job must re-execute, all its direct dependents are marked stale, and this propagation continues until all transitively affected jobs are identified.

Snakemake evaluates each rule against its *direct* inputs and outputs. In some configurations, if an intermediate file is deleted but a downstream output still exists, Snakemake may report "Nothing to be done." Snakemake 7+'s recorded provenance (`--rerun-triggers`) governs re-execution on code, parameter, and input-set changes, but the existence check remains per-rule and local. OxyMake's cache pre-scan avoids this issue: it verifies that every declared output still exists on disk, so a deleted intermediate always triggers a transitive rebuild. For example, after deleting a single intermediate file in a multi-stage pipeline, `ox run` rebuilds that stage and all downstream stages (preprocess → merge → analyze → report). A minimal reproduction comparing both tools is in the public repository under `examples/intermediate-deletion/`. The property this cascade enforces—no downstream job observes an output whose producer has been invalidated but whose cache entry has not been retired—is defended by the implementation's cache pre-scan and its integration tests; a dedicated specification of the eviction race is outside the evaluated scope (§3.5).
Concurrent sessions on *disjoint* job sets are the supported pattern today:

```
# Terminal 1: dataset-A pipeline
ox run --where dataset=train

# Terminal 2: dataset-B pipeline (concurrent)
ox run --where dataset=test
```

For *overlapping* job sets, the state layer implements a persistent claim-state protocol: atomic SQL updates with optimistic locking (an `UPDATE...WHERE status='pending'` that affects zero rows means another session claimed the job first), session heartbeats, and a two-phase reclaim that resets the jobs of stale sessions (older than 2 minutes) to pending. `CooperativeClaim.tla` model-checks this protocol, which is not yet the scheduling gate (§3.2).

Five commands share the convergent model and one filter surface: `ox run` (converge), `ox cancel` (abort), `ox invalidate` (reset), `ox plan` (preview), and `ox status` (observe). All five commands accept `-where`, `-rule`, and `-json` flags.

4.5 Tags, Guards, and Incremental Growth

Large, evolving workflows are managed through tags, conditional guards, and incremental growth.

Tags. Every resolved wildcard automatically becomes a tag (implicit tags). Rules can also declare explicit tags for cross-cutting concerns (e.g., `stage = "features"`, `cost = "high"`). The `--where` filter selects target jobs by tag values, then backward-chains to include all necessary upstream dependencies. Tags also enable **hierarchical DAG visualization** via

`ox dag --group-by stage`, which collapses jobs into meta-nodes by tag grouping—keeping very large DAGs comprehensible at a glance.

Conditional guards. The `when` clause enables non-uniform DAG branches where some wildcard instances need sub-workflows that others do not:

```
[rule.spectral_analysis]
input = ["results/{sample}.parquet"]
output = ["diagnostics/{sample}/outliers.png"]
when = "sample in @high_variance_samples"
```

Guards are evaluated at DAG resolution time—a job whose guard is false is never created in the graph. This enables instance-specific branching without phantom nodes.

Incremental growth. Research workflows grow iteratively: explore, select, evaluate, refine. OxyMake supports this through content-addressable incrementality (adding rules never invalidates existing results), workflow composition via `include` directives, snapshots for milestone comparison (`ox snapshot diff baseline-v1`), and run annotations (`ox run –note "Testing new preprocessing step"`) recorded in the state database alongside each run's metrics.

5 Implementation

5.1 Rust as Implementation Language

Rust was chosen because its properties directly serve OxyMake's design goals.

Type system as architecture. Rust's type system enforces architectural invariants at compile time. `Rule` (unresolved wildcards) and `ConcreteJob` (fully resolved) are distinct types, so an unresolved rule cannot be passed where a scheduled job is expected. `Send + Sync` bounds enforce thread safety for the concurrent scheduler at compile time. `Result<T, E>` on fallible functions makes ordinary error propagation explicit rather than exceptional, so a failing job returns an error the scheduler handles instead of unwinding the process. The scheduler cancels in-flight jobs on shutdown or panic, sending `SIGTERM` to each process group to clean up child and grandchild processes; a `SIGKILL` to the scheduler itself, or a kernel-level failure, bypasses this path.

Performance ceiling. Three properties bound the engine's speed. The workflow format is inert: TOML parsing takes microseconds, where a Python-DSL workflow incurs interpreter import time on every invocation. The graph algorithms are asymptotically cheap: the `petgraph` library [30] provides an $O(|V| + |E|)$ topological sort via Kahn's algorithm, and the `ProducerIndex`'s precompiled-pattern resolution is $O(R \times P)$ in the rule count R and the output-pattern count P, with the regex-construction constant paid once (§6.4). The measured range extends to 10,001 jobs (Table 6); extrapolating the observed near-linear slope to 5×10^4 and 10^5 jobs is a projection, not a measurement (§6.4). We expect hashing not to be the bottleneck: BLAKE3 [27, 28] is reported at over 6.9 GiB/s single-threaded on x86-64, above the I/O rate of the files a typical workflow reads. We have not profiled the hash phase of our benchmark, so this expectation rests on published throughput figures, not on a measurement of this system.

Distribution. OxyMake ships as one self-contained `ox` executable: it bundles no interpreter and requires no OxyMake daemon or service, linking only the host platform's system libraries (we do not claim a fully static link, which would require a per-target linkage attestation we have not published). Cross-compilation targets Linux, macOS, and Windows. Installation requires only `cargo install oxymake` or downloading a pre-built binary. The workspace compiles via standard `cargo build --release`.

5.2 Testing and Documentation

Testing. 1,928 tests accompany 64,596 SLOC (Table 5); test density varies widely between the engine crates and peripheral ones. The testing stack is `cargo test` for unit and doc tests, `cargo-llvm-cov` for line coverage, `proptest` for property-based testing of DAG invariants and cache-key stability, and `insta` for snapshot testing of TOML parsing and CLI output.

Implementation metrics are reported in Table 5.

Literate programming. Public functions and types carry `///` doc comments with executable examples that compile and run as part of `cargo test`. The 63 doc tests are documentation, usage examples, and regression tests at once. Because they run in CI, a documented example that stops compiling fails the build, which bounds—rather than eliminates—documentation drift for the examples under test. Prose documentation and design rationale in module headers and Architecture Decision Records (ADRs) are not machine-checked. The project includes 109 documentation files covering concepts, command references, format specifications, and error indices.

5.3 Crate Architecture

OxyMake is organized as a Cargo workspace of 25 crates (24 workspace members plus `ox-metrics`, which is workspace-excluded for dependency isolation), each with a single, well-defined responsibility. Table 4 summarizes the architectural decomposition, and Table 5 provides concrete implementation metrics collected from the codebase.

The size distribution reveals the expected concentration: `ox-core` contains 24% of the codebase (DAG algorithms, scheduler, type system, event bus) and 29% of the unit and integration tests. Three stub crates (`ox-env-system`, `ox-env-uv`, `ox-storage-local`) contain trait scaffolding only; the functionality they will host is implemented inline today (§4.3). Figure 3 illustrates the dependency relationships between crates.

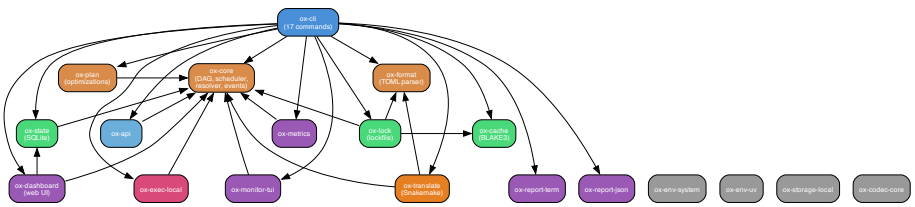

Figure 3: Dependency graph of the principal crates in OxyMake's workspace. `ox-cli` sits at the top as the entry point, delegating to `ox-core` (engine), `ox-format` (TOML parser), `ox-state` (SQLite persistence), and specialized crates for execution, reporting, monitoring, and translation.

Crate	Responsibility
ox-core	DAG, scheduler, traits, types, events
ox-format	TOML parsing of Oxymakefile
ox-state	SQLite persistence
ox-cache	Content-addressable hashing (BLAKE3)
ox-plan	Optimization passes on JobGraph
ox-api	Public Rust API facade *(scaffolding; see below)*
ox-exec-*	Executor backends (local, Slurm, Ray)
ox-cache-remote	Remote cache backend (directory; S3/GCS planned)
ox-mcp	Model Context Protocol (MCP) server
ox-storage-*	Storage backends (local; S3 planned)
ox-env-*	Environment providers (system, uv, conda, Docker)
ox-codec-*	Format codecs (CSV, JSON, Parquet, columnar)
ox-report-*	Reporters (terminal, NDJSON)
ox-lock	Reproducibility lockfile (BLAKE3)
ox-translate	Multi-format workflow translator (Snakemake, WDL)
ox-dashboard	Web dashboard server with DAG visualization
ox-monitor-tui	Terminal UI monitor
ox-metrics	Prometheus metrics export
ox-render	Terminal styling and semantic color roles
ox-cli	CLI binary (clap wrapper over the engine crates)

Table 4: Crate responsibilities in the OxyMake workspace.

Each crate has a strict boundary: ox-core never performs file I/O or network calls; ox-format never validates file existence or executes rules; ox-state never decides whether a job should re-run. These boundaries are enforced by Cargo dependency rules—a crate cannot access functionality it does not depend on.

5.4 Machine-Readable Execution Interface

Every ox command supports -json for structured output. When active, events are emitted as newline-delimited JSON (NDJSON) on stdout:

```
{"type":"run.started",
 "total_jobs":10001,
 "to_run":847,"cached":9154}
{"type":"job.completed",
 "id":"align_S001","duration_ms":272000}
{"type":"gate.reached",
 "id":"qc_check",
 "message":"Review QC metrics"}
```

A downstream tool reads line-by-line, reacting to typed events. Gate approval is programmatic:

```
ox gate approve qc_check \
  --approver "ci:qc-runner" \
  --reason "metrics within threshold"
```

For Rust embedding, the scheduler exposes a typed tokio::broadcast channel of Event values, enabling in-process integration without stdout parsing.

Crate	Lines	Tests	Doc Tests
ox-core	15,568	550	29
ox-cli	11,392	211	0
ox-translate	6,170	162	2
ox-exec-ray	5,383	156	1
ox-exec-local	4,602	103	1
ox-state	3,862	70	10
ox-format	3,497	145	0
ox-exec-slurm	3,382	92	1
ox-cache	2,279	99	0
ox-mcp	1,901	85	0
ox-dashboard	1,154	32	1
ox-monitor-tui	1,049	23	3
ox-report-term	926	30	3
ox-plan	681	25	6
ox-lock	680	11	0
ox-api	473	10	2
ox-cache-remote	455	15	0
ox-report-json	389	20	1
ox-metrics[†]	277	8	3
ox-codec-core	275	20	0
ox-render	176	6	0
Stub crates (ox-env-*, ox-storage-local)	25	3	0
oxymake (name reservation)	0	0	0
Total	**64,596**	**1,876**	**63**

Table 5: Crate architecture with implementation size and test counts. Lines are Rust code lines (tokei); Tests are passing unit and integration tests per crate (cargo test); doc tests are listed separately. Columns sum to the Total row. [†]ox-metrics is excluded from the Cargo workspace for dependency isolation and is tested via its own manifest; the workspace test run therefore reports 1,928 tests (the table totals minus ox-metrics's 8 tests and 3 doc tests).

MCP server. A subset of these operations is exposed over the Model Context Protocol through `ox serve -mcp`, an stdio server (crate `ox-mcp`). The tool catalogue is deliberately narrow: eight tools, of which seven are read-only inspection—`ox_status`, `ox_plan`, `ox_dag`, `ox_logs`, `ox_history`, `ox_lint`, `ox_explain`—plus one mutating tool, `ox_clean`, which removes cached artifacts and state. Notably there is *no* `ox_run` tool and no gate-approval tool: an agent can plan, inspect, and explain a workflow over MCP, but starting execution and approving a gate remain human- or script-driven through the CLI. This is a conservative default rather than a limitation of the protocol, and the catalogue is pinned by a test so it cannot widen silently. The server is an interface, not a durability layer: it adds no queueing, retry, or cross-host recovery, and an agent driving external side effects must supply its own idempotency and approval. We report the interface as implemented and do not evaluate agent task performance over it.

5.5 State Management

Three logically separated state concerns share a physical SQLite database (`.oxymake/state.db`):

Execution state (ephemeral): What is running/pending/done. Reconstructible from the DAG and cache if lost. Uses WAL mode and atomic transactions for concurrent access.

Cache state (persistent, immutable): Content-addressable store keyed by Equation 1. Directory structure: `.oxymake/cache/{prefix}/{hash}`. The artefact bytes are independent of SQLite and can be shared across same-platform machines through the directory remote cache (`ox run -cache-remote <dir>`), which verifies every restored artefact by content (index locality: §3.1). S3/GCS transports are not implemented (§6.8).

Audit state (append-only): Run history with notes, per-job metrics (wall time, peak memory, exit code, hostname, environment hash). This provenance record enables reproducibility audits and performance regression detection. It is impossible to backfill if not collected from run 1.

Schema versioning uses `PRAGMA user_version` with automatic migrations from the first release; a migration runs inside a transaction, so on local disk—where the `StateDbAtomicCommit` axiom holds (§3.5)—a `state.db` is never left half-migrated. On a shared filesystem that premise fails and the property is not claimed (§7.3).

6 Evaluation

We evaluate OxyMake along seven axes: performance benchmarks on synthetic workflows (Section 6.1), the content-addressing behaviour under mtime churn (Section 6.3), a scalability study with algorithmic optimization (Section 6.4), Snakemake compatibility (Section 6.5), startup overhead (Section 6.6), binary footprint (Section 6.7), and FAIR compliance (Section 6.9). All experiments were run on a single Apple M4 Max (16-core, 128 GB RAM) running Darwin 25.5.0 (arm64); no Linux/x86_64 measurements were taken, and no cross-architecture claim is made. The DAG-resolution and end-to-end head-to-head numbers (Tables 6 and 7) come from a single head-to-head benchmark harness, `bench/snakemake-vs-oxymake/` in the public repository: the sub-second resolution phase is timed with `hyperfine` (median over its

statistical runs), and the minutes-scale end-to-end phase with `/usr/bin/time` as the median of three runs. Dispersion (IQR or confidence intervals) is not reported; the statistical limits of the evaluation are stated in §7.3. The startup and binary-footprint micro-benchmarks were timed separately as noted in their subsections.

6.1 DAG Resolution Performance

All numbers in this section come from a single benchmark harness in the public repository, `bench/snakemake-vs-oxymake/`, with a one-line reproducer (`bash run.sh` from that directory). The harness drives a synthetic four-layer DAG (`seed` $\rightarrow N$ `gen` shell rules $\rightarrow N$ `process` Python-via-shell rules $\rightarrow N$ `finalize` file-copy rules $\rightarrow$ `merge`), totalling $3N+2$ jobs; the three scales use N=33, 333, and 3333, i.e. exactly 101, 1,001, and 10,001 jobs, and the tables label them by these exact counts. The same DAG and per-job work are declared once as `workflow.toml` (OxyMake) and once as `workflow.smk` (Snakemake 7.32.4); only the orchestrator changes between runs. We benchmark the Snakemake 7.x line because it is the version that added recorded provenance to the mtime comparison, and is therefore the strongest change-detection comparator; the 8.x line changes the executor and deployment model but not the provenance-based rebuild decision this paper compares against. DAG resolution time was measured using `ox plan` for OxyMake and `snakemake -dryrun` for Snakemake—both parse the workflow, resolve all wildcards, build the job graph, and report the execution plan without running any jobs. The resolution phase is sub-100 ms, so we time it with `hyperfine` (statistical warmup, no subprocess-wrapper overhead); the end-to-end phase below is minutes-scale and timed with `/usr/bin/time`. One scope note applies to every head-to-head number in this section and the next, naming exactly which validation mode produced it. DAG resolution is not symmetric on this point: `ox plan` exercises no cache validation, whereas building the DAG in `snakemake -dryrun` evaluates the rerun triggers for every job unconditionally, so the dry run stats inputs and outputs, reads `.snakemake/metadata` and checksums eligible inputs. The ratio therefore bounds rather than isolates the resolution difference. The cold and warm end-to-end rows run Snakemake under its default rerun-triggers (recorded per-output provenance) and OxyMake under `--cache-validation=mtime`, the metadata fast path (§3.1); an additional warm run uses `--cache-validation=hash`, full content re-verification. The current default, `mtime+hash`, was not measured separately; it follows the same metadata fast path on an undisturbed tree. The measurements that back the content-addressing thesis are that `hash`-mode row and the `git checkout` scenario—every mtime perturbed, no content changed, count the jobs each engine re-runs—reported in §6.3.

Jobs	OxyMake `ox plan`	Snakemake `-dryrun`	Speedup
101	4 ms	418 ms	101.9×
1,001	10 ms	512 ms	50.7×
10,001	69 ms	2,310 ms	33.3×

Table 6: DAG-resolution wall-clock time, median over hyperfine runs (benchmark harness `bench/snakemake-vs-oxymake/`). Measured 2026-06-10, Apple M4 Max (Darwin 25.5.0, 16 cores); OxyMake `ox plan`, Snakemake 7.32.4 `snakemake -dryrun`. Speedups are computed from the unrounded measurements; the times shown are rounded. Raw measurements and the reproducer are in `bench/snakemake-vs-oxymake/RESULTS.md` in the public repository.

OxyMake resolves a 10,001-job DAG in 69 ms versus Snakemake's 2.31 s, a 33.3× difference on

DAG resolution. The ratio *narrows* with scale (101.9× at 101 jobs to 33.3× at 10,001) because Snakemake amortises its fixed Python-interpreter startup over more jobs while OxyMake's resolution grows linearly: from 101 to 10,001 jobs (≈100× more work) OxyMake adds ≈65 ms, a marginal cost of ≈6.6 μs/job, consistent with a per-target lookup whose cost is a handful of precompiled regex matches at this rule count (Section 6.4). **This comparison is scoped to DAG resolution**; the end-to-end comparison, in which OxyMake is slower on a cold run, is reported next.

6.2 End-to-End Execution

DAG resolution is the phase OxyMake optimises; it is not the whole story. The same harness also measures *end-to-end* wall time—resolve plus execute every job—and here OxyMake is *slower* than Snakemake on a cold run.

Jobs	Snakemake	OxyMake	OxyMake / Snakemake
101	1.10 s	1.37 s	1.25× (slower)
1,001	4.31 s	9.74 s	2.26× (slower)
10,001	1.6 min	2.4 min	1.44× (slower)

Table 7: Cold end-to-end wall time (same benchmark harness). OxyMake runs 1.25–2.3× slower than Snakemake on a cold execution across scales. Ratios are computed from the unrounded measurements; the times shown are rounded. Warm-cache re-run is the inverse (next paragraph).

On a first, cold execution OxyMake performs work Snakemake does not: it hashes every rule's source, inputs, parameters, environment and platform into a BLAKE3 cache key and writes a content-addressed store and an `ox.lock` audit record. A first cold run has no prior provenance for either engine to consult. We attribute the measured cold differential to these content-store and hash writes; this attribution is a hypothesis consistent with the design, not the result of a phase-level profile, which we have not performed. The overhead is real wall-clock cost on the cold path, exchanged for content-addressable correctness, cache-key validity across same-platform machines, and an auditable rebuild decision (§6.9). Three measurements on the *same* harness show what that bookkeeping provides:

- **Warm-cache re-run** (the common inner-loop case): at 10,001 jobs OxyMake completes a no-op rebuild in 372 ms versus Snakemake's 2.81 s—7.54× faster. That row is the `mtime` metadata fast path (the `mtime+hash` default takes the same path on an undisturbed tree); under `-cache-validation=hash`, which re-hashes every input and output before deciding, the same no-op rebuild takes 698 ms—still 4.02× faster than Snakemake, with every byte verified.
- **Minimal-rebuild correctness**: rewriting the content of one Layer-1 input, both systems re-run exactly the 3 affected jobs at every scale—OxyMake's content-addressed decision is as tight as Snakemake's provenance decision, and its computation key is identical on any machine of the same platform where a per-tree record is bound to one tree (the platform term in Eq. 1 scopes the key to one OS/arch pair; heterogeneous reuse is future work, and the rebuild-decision preconditions are stated in §1).
- **Memory**: peak resident set of the orchestrator on the cold 10,001-job run is 90.7 MiB for OxyMake versus 184.7 MiB for Snakemake—2.04× smaller.

In summary, on this workload OxyMake is 1.25–2.3× slower on cold runs and 4.02–7.54× faster on warm no-op runs, depending on validation policy. These results should not be generalized beyond the evaluated workload. Section 6.4 treats DAG-resolution scaling; the FAIR contract that the cold-path bookkeeping underwrites is evaluated in Section 6.9.

6.3 Content-Addressing under mtime Churn: the git-checkout Test

The scenario that separates timestamp-trusting from content-checking validation is mtime churn: every timestamp moves, no byte changes. The harness reproduces it by bumping the mtime of a shared tracked input (`bench_lib.py`, a declared `lib` input of every `process` job) after a clean build. This models the mtime-only perturbation that a `git checkout`, a tree copy, or a backup-restore introduces: metadata moves, content does not. Those operations differ in other respects; the benchmark isolates the mtime change they share. A decision that trusts timestamps must re-run every job that reads the file ($2N+1$: every `process`, every `finalize`, plus `merge`); a decision that checks content must re-run zero.

Jobs	Snakemake 7.32.4	OxyMake `mtime`	OxyMake `hash`
101	0	67	0
1,001	0	667	0
10,001	0	6,667	0

Table 8: Jobs re-run after bumping a shared input's mtime without changing a byte (same benchmark harness, measured 2026-06-10). Validation modes are explicit: Snakemake runs its default rerun-triggers (recorded provenance); the OxyMake columns pin `--cache-validation` to `mtime` and `hash` respectively. The shipping `mtime+hash` default is not a separate measured column; the two measured modes bracket it (see text).

Pure `mtime` validation re-runs the full radius. It re-runs $2N+1$ jobs at every scale (6,667 at the largest), because the metadata-only policy re-runs every job whose timestamp changed. The `mtime+hash` policy re-hashes a file whose metadata moved before deciding, so a churned tree re-runs zero jobs; the only cost is the re-hash of the perturbed files—a warm-run cost that falls between the two measured endpoints at 10,001 jobs, 372 ms (metadata only) and 698 ms (re-hash everything); it was not measured as a separate condition.

Snakemake 7.32.4 re-runs zero jobs. It re-ran zero jobs even with the input's mtime forced to the year 2030, under an explicit `--rerun-triggers mtime`. The mechanism is not that live timestamps are ignored—they are compared first—but that an input newer than the oldest output counts as updated only if its recorded SHA-256 also fails to match. The perturbed file is 638 bytes, below the 100,000-byte threshold above which Snakemake records no checksum, so the checksum decides here; a larger shared input would leave the mtime comparison to decide alone. The phantom-re-run failure mode is real for GNU Make and for any engine's pure-mtime fast path, but the benchmarked Snakemake version does not exhibit it.

Content-addressing does not improve this scenario on one machine. On a single working tree, `hash` mode reaches parity with Snakemake's provenance, not superiority. What a working-tree-bound provenance record cannot offer is: a caching decision that is a pure function of content and workflow-root-relative paths, hence a computation key that is identical on any same-platform machine (§3.1); detection of on-disk output corruption that a provenance record would serve stale (§3.1); and a plan and key derivation auditable from the `ox.lock`

record, the rebuild decision itself being auditable given the checkout's live reuse state—local key-to-output index, materialised outputs, and validation policy (§1, §6.9). It also avoids the pure-`mtime` re-run radius the middle column documents.

6.4 Scalability: ProducerIndex Optimization

The measured resolution times in Table 6 ($\leq$69 ms at 10,001 jobs) are already small, but the naive backward-chaining algorithm parses and compiles every candidate output pattern for every target it resolves—$O(R \times P)$ pattern compilations, where R is the number of rules and P the number of output patterns, with a large constant dominated by regex construction. We implemented a `ProducerIndex`: all output patterns are parsed and compiled *once*, up front, so each target lookup is a scan of precompiled regexes rather than a parse-and-compile cycle. The asymptotic complexity remains $O(R \times P)$—lookup is still a linear scan over the compiled patterns—but the constant drops substantially, since per-target regex compilation was the dominant cost at scale. A hash-based prefix index that would reduce lookup to amortized $O(R + P)$ is future work, not part of the measured system. The scaling observed across $101 \rightarrow 10{,}001$ jobs in Table 6 (4 ms $\rightarrow$ 10 ms $\rightarrow$ 69 ms: $\approx$65 ms added for a $\approx$100$\times$ increase in job count, $\approx$6.6 μs/job) is near-linear in job count because the rule count—and hence the per-target scan cost—stays small and fixed while the job count grows; quadratic behaviour would surface only if rules and targets grew together. Beyond the measured range we report only projections: extrapolating the slope suggests $\sim$0.3 s at 5×10^4 jobs and sub-second at 10^5 jobs. **These are algorithmic-headroom projections, not measurements**—the harness was run to 10,001 jobs only; 5×10^4 and 10^5 were not measured.

6.5 Workflow Language Compatibility

Translation is **bidirectional** and supports multiple source formats. The `ox translate` command converts both Snakemake and WDL workflows to Oxymakefile TOML, while `ox export snakemake` and `ox export wdl` convert an Oxymakefile back to the respective formats. Source format is auto-detected from file extensions (`.smk`, `.wdl`) or content inspection.

Snakemake. We exercised Snakemake translation on four workflows of increasing complexity: an eight-rule tutorial pipeline over three samples, a six-rule RNA-seq counting workflow over four samples driven by a `configfile`, a six-rule CSV ETL pipeline over three sources, and a five-rule multi-sample QC workflow over five samples. The workflows, their translated Oxymakefiles, and the full result log are in the public repository under `benchmark/snakemake-compat/` (see `RESULTS.md` there). The equivalence oracle is the output file tree: the translated Oxymakefile produced the same set of output paths as the Snakemake source. This checks structural translation, not byte-identical outputs of nondeterministic commands.

None of the four ran unmodified. All four executed to completion only *after* manual fixes to the translated Oxymakefile, between three and five edits per workflow. The recurring interventions were: adding `expand = "product"` to rules that aggregate over wildcards (the translator escalates `expand()` calls that reference a Python-scoped variable instead of resolving them); replacing `{input.name}` with unnamed `{input}` in aggregation rules, because a named input under `expand` substitutes only the first matching path; rewriting

Snakemake's {{}} brace escaping; replacing bash-only shell constructs, since OxyMake executes under /bin/sh; and naming targets explicitly, because rules are stored in a BTreeMap so the default target is the alphabetically first rule rather than rule all. Two of the four translated with no escalation at all and still needed four hand edits each, so a clean translation report is not evidence of a runnable workflow.

Translation handles rule definitions, wildcard expansion, params/threads/resources blocks, wildcard_constraints, and configfile references. Unsupported Snakemake features (embedded Python expressions, run: blocks with arbitrary code, dynamic thread counts, wrapper, shadow/group/envmodules) emit structured warnings or escalations with migration guidance. The honest summary is that ox translate is a migration aid that removes most of the mechanical work, not a drop-in transpiler.

WDL (Workflow Description Language). WDL [37] is the standard workflow language in genomics, used with Cromwell, miniWDL, and the Terra platform. OxyMake's WDL translation maps WDL's task/workflow/call model to OxyMake's rule-based model: each WDL task becomes an OxyMake rule, command blocks become shell directives, and runtime blocks (docker, cpu, memory, disks) map directly to OxyMake's resource and environment declarations. WDL's scatter blocks are translated to OxyMake's expand mode, and WDL placeholder syntax (~{var} and ${var}) is converted to OxyMake's {var} format. WDL's richer type system (File, Array[File], Int) produces structured escalations for constructs that require manual review, such as optional inputs (File?) and glob expressions.

Why TOML, not WDL? WDL is optimized for a specific domain: bioinformatics pipelines running on cloud infrastructure via engines like Cromwell. Its typed, portable specification is well suited to genomics workflows but requires declarations that general computational pipelines do not need: every input and output must carry an explicit type declaration, and the workflow/call/scatter hierarchy adds structural overhead for simple file-to-file transformations. (The runtime section and its individual attributes are *optional* in WDL 1.2.) OxyMake's TOML format is domain-agnostic— a three-line rule with input, output, and shell suffices for simple cases, while the same format scales to complex multi-wildcard pipelines with resource declarations. OxyMake ships as one ox executable requiring no OxyMake daemon for local execution; WDL requires a separate execution engine (Cromwell, miniWDL). The bidirectional translator bridges both ecosystems: bioinformatics teams can import their existing WDL workflows into OxyMake for local development and export back to WDL for cloud execution on Terra/Cromwell.

6.6 Startup Time

We measured cold-start time for three commands of increasing complexity:

Command	Wall-clock time
ox -help	6 ms
ox lint (10 rules)	7 ms
ox lint (1,000 rules)	7 ms

Table 9: Startup time (median of 3 runs).

Startup time is 6–7 ms regardless of workflow size (median of 3 runs), so TOML parse time is

negligible relative to binary startup. The native Rust binary carries no Python or JVM startup overhead. These numbers are a component measurement rather than a usability comparison.

6.7 Binary Size

Metric	Value
Binary size (release, default features)	14.9 MB
Binary type	Mach-O 64-bit arm64
Full release build time	71 s

Table 10: Binary footprint and build metrics.

The 14.9 MB binary includes all default-feature crates (among them the TUI monitor, web dashboard, and Snakemake/WDL translator). It is one self-contained executable that bundles no interpreter and needs no OxyMake daemon, linking only the host platform's system libraries; installable via `cargo install` or direct download.

6.8 Implementation Status

Table 11 is an implementation matrix: it lists the features that are implemented and marks which are exercised by the benchmark harness. Implemented and evaluated are distinct: the Slurm and Ray executors, the MCP server, the dashboard, and the directory remote cache are implemented but not measured in this evaluation (§6).
Kubernetes execution, object-store (S3/GCS) cache backends, in-memory data passing on the local executor, and five of the six optimization passes are not implemented and are excluded from the evaluation.

6.9 FAIR Compliance Assessment

We assess OxyMake against the FAIR workflow indicators defined by Goble et al. [14] and operationalized by Wilkinson et al. [38], and against the FAIR4RS principles for the engine itself [7]. Table 12 is a self-assessment: each row records the mechanism that supports the indicator, not an external audit or a third-party certification. It mixes properties of the engine, the workflow format, and the produced artefacts, and a reader should read each cell as "the mechanism is present", not "the indicator is independently verified".
Under this self-assessment OxyMake supports three indicators natively (A2, R1, R2), six partially, one is not assessed (I3), and one is future work (F3). The largest gaps are the absence of a workflow registry for discovery (F3) and the fact that the TOML workflow format is not a community standard like CWL (I2), though the `ox translate` command provides bidirectional conversion with both Snakemake and WDL as bridges to the broader bioinformatics and genomics ecosystems.

Feature	Notes
TOML-based workflow definition	Oxymakefile.toml parsing
Backward-chaining DAG resolution	Wildcards, fan-out
Content-addressable caching	BLAKE3 + mtime fast-path
Three-graph architecture	Rule → Job → Exec
Cache-pruning optimization pass	Skips up-to-date jobs
Four execution modes	shell, run, script, call
NDJSON event API (`-json`)	All commands
Gate approval	`ox gate approve/reject`
Idempotent convergent execution	SQLite coordination
Tags and `-where` filter	Implicit + explicit tags
Conditional guards (`when`)	DAG-time evaluation
Lockfile (`ox.lock`)	Re-derivable plan of record
Snakemake translator	`ox translate`, `ox export snakemake`
WDL translator	`ox translate`, `ox export wdl`
TUI monitor (`ox top`)	Real-time dashboard
Web dashboard	HTML status page
Slurm executor	`-executor slurm`
MCP server	Model Context Protocol endpoint
Ray executor	`-executor ray`
Directory remote cache	`-cache-remote`, blob store (§3.1)

Table 11: Implementation matrix. All rows are implemented; the benchmark harness exercises TOML parsing, DAG resolution, content-addressable caching, cache pruning, and the `shell` execution mode on the local executor (its Python and file-copy work is launched through shell rules). The `run`, `script`, and `call` modes are covered by the test suite but not by the benchmark. The remaining rows are implemented but not measured here.

Principle	Indicator	OxyMake	Mechanism
Findable	F1: Unique ID	Partial	Lockfile content hash; not a registry-backed persistent identifier
	F2: Rich metadata	Partial	Machine-readable TOML; no standard metadata vocabulary
	F3: Searchable registry	Future	Workflow registry planned
Accessible	A1: Standard protocol	Partial	Files retrievable via Git/HTTP; artefact accessibility not guaranteed
	A2: Open format	Native	TOML, no vendor lock-in
Interoperable	I1: Standard serialization	Partial	Data artefacts in Parquet/CSV/IPC; workflow language engine-specific
	I2: Workflow language	Partial	TOML + WDL/Snakemake bridge
	I3: Cross-platform	Not assessed	Compiles for three OSes; only macOS/arm64 exercised here
Reusable	R1: Provenance	Native	Audit trail in state.db
	R2: Reproducibility	Native	Lockfile + content cache (assumes declared inputs, §3.1)
	R3: Community standards	Partial	Open licence; no community packaging standard (e.g. RO-Crate)

Table 12: FAIR self-assessment. "Native" = the indicator is supported by a built-in mechanism; "Partial" = a mechanism exists but does not fully satisfy the indicator as defined by the cited papers; "Not assessed" = no evidence either way in this evaluation; "Future" = not yet implemented.

Benchmark	Result
DAG resolution @ 1,001 jobs	10 ms
DAG resolution @ 10,001 jobs	69 ms
DAG res. speedup @ 10,001 jobs	33.3×
Startup time (`ox --help`)	6 ms
Binary size (default)	14.9 MB
Test suite	1,928 passing

Table 13: Summary of the measured results (§6.1–§6.7).

6.10 Performance Summary

7 Discussion

7.1 Positioning in the Landscape

Table 14 places OxyMake against neighbouring systems on four axes: the authoring surface, how the rebuild decision is made, how the system is deployed, and whether it exposes a machine-readable interface. No listed system combines all four properties OxyMake combines—a file-rule surface, a content-derived computation key as the default identity, a single daemon-free binary, and a structured machine interface—though each property individually appears in some neighbour.

System	Surface	Rebuild decision	Deployment	Machine interface
GNU Make	file rules	live mtime	binary	no
Snakemake 7	file rules (Py)	per-tree provenance	Python runtime	partial
DVC	pipeline stages	content hashes	Python runtime	partial
Nextflow	channels (Groovy)	path/size/mtime	JVM runtime	partial
cwltool + Arvados	CWL standard	content-sensitive cache	platform	varies
Bazel / Buck2	build targets	content-addressed	daemon	partial
Airflow	DAG-as-code	none (task re-runs)	service	yes
Prefect / Dagster	DAG-as-code	inputs + code version	service	yes
Dask / Ray	task/actor API	(execution substrate)	runtime/cluster	API
OxyMake	file rules (TOML)	content key + index	single Rust binary	yes (NDJSON, MCP)

Table 14: Positioning on four axes. The rebuild-decision column names what each engine's default decision reads; for OxyMake the content-derived key is the checkout-independent identity of a computation, and the decision additionally consults the local key-to-output index, the materialised outputs, and the validation policy (§3.1). Slurm is not a row: it is an execution substrate OxyMake targets (`-executor slurm`), not a competitor. Dask and Ray are likewise substrates the executor backends consume rather than engines OxyMake replaces. Snakemake's opt-in `-cache` adds a content-keyed between-workflow cache beside the per-tree record without changing the default rebuild decision (§2.1); DVC versions pipeline stages rather than expanding wildcard file rules (§2.1).

Compared to Snakemake, OxyMake preserves the backward-chaining DAG paradigm while replacing its rebuild decision—live mtimes in the Make tradition, or working-tree-bound provenance records in Snakemake 7, which are recorded rather than heuristic—with content-addressable caching (including transitive invalidation of deleted intermediates; Section 4.4), the Python DSL with declarative TOML, and the Python runtime with a Rust engine. Compared to the guix-cwl stack [31, 39], OxyMake targets the orchestration layers (L2–L4) of the FAIR ladder (Table 1) and leaves the binary substrate (L1) to a content-addressed package manager; whether the two contracts compose cleanly is a future direction (§7.4), not a claim made here. Compared to service-deployment orchestrators, OxyMake retains the simplicity of file-based rules while adding content-addressable caching and a machine-readable API. Compared to build systems (Bazel, Buck), the difference is semantics per axis rather than presence versus absence of whole features: Bazel has target patterns, configurable toolchains and platform constraints, and Starlark extension points for policy, so it would be wrong to say it lacks wildcards, environment management, or gates. What differs is what each maps onto—OxyMake's wildcards expand *output filenames* into per-path jobs rather than selecting build targets; its

environment declarations name a scientific runtime (conda YAML, `uv.lock`, an image) hashed by content into the job key rather than a toolchain resolved by constraint; and its gates are run-time human or agent approval checkpoints inside a workflow, not analysis-time policy checks.

In the Mokhov et al. taxonomy [23, §3], OxyMake combines a *topological scheduler* (task order fixed before execution from the dependency graph) with *verifying traces* (skip a task when its recorded input/output hashes still match disk) augmented by content-addressing. All dependencies are resolved statically before execution, placing OxyMake in the same scheduler column as Make, Ninja and Buck rather than the restarting column that holds Bazel or the suspending column occupied by Shake. Unlike Make, OxyMake replaces dirty-bit rebuilding (re-run whenever any input timestamp changes) with content-addressed verifying traces, and unlike Bazel, it targets scientific workflows with declarative TOML rules, wildcard expansion, and environment management.

7.2 Intended Use

The design fits teams that already express computation as deterministic commands with files as artefact boundaries and want to avoid recomputation without deploying a service. Two grounds are the most credible first targets. Portable scientific and HPC pipelines have a documented pain: on shared filesystems the cache and `$HOME` may be unwritable or carry inconsistent timestamps, and per-tree provenance does not survive a fresh checkout [11]. Small data teams running a modest batch DAG face a deployment cost in the service-oriented alternatives, which are built around a persistent scheduler and its supporting services [6]; that infrastructure is disproportionate to a single batch job. ML feature-and-sweep pipelines and agent-driven runs are plausible fits by the same reasoning, but are not demonstrated here: OxyMake integrates alongside experiment trackers rather than replacing them, and the MCP interface is a driving surface, not a durability layer (§5.4). These are design-fit inferences from the engine's contract, not adoption results.

7.3 Limitations and Future Work

TOML expressiveness. The decision to use TOML rather than a Turing-complete DSL limits the expressiveness of workflow definitions. Complex configuration generation must happen outside the Oxymakefile via scripts. While this preserves static parseability, it may frustrate users accustomed to Snakemake's Python flexibility. We note that the choice is not strictly binary: languages such as Starlark [5] (deterministic, sandboxed Python subset used by Bazel), CUE [36] (constraint-based configuration with types and validation), and Dhall [15] (a total functional language for configuration) occupy a middle ground between inert data formats and general-purpose languages, offering controlled expressiveness with static analysis guarantees. OxyMake opts for the simplest end of this spectrum—plain TOML—because workflow *specification* rarely needs computation; when it does, a config generation pattern (`python gen_config.py > config.toml`) or a minimal expression language (pure functions, no loops) provides an escape hatch without compromising the static parseability of the Oxymakefile itself.

No sandbox: undeclared inputs are trusted. As stated in the threat model (§3.1), OxyMake does not isolate rule execution, so the cache key's completeness is only as good as the user's input declaration: an undeclared read is a silent stale-reuse hazard. An opt-in sandboxed

execution mode (namespace or seatbelt isolation that turns an undeclared read into a build-time error, recovering Nix's enforcement property) is future work; until it lands, the mitigation is declaration discipline plus the auditable `ox.lock` record.

SQLite on network filesystems. SQLite WAL mode does not work on NFS, Lustre, or GPFS—precisely the filesystems used on HPC clusters where Slurm runs. OxyMake requires `.oxymake/` to reside on local disk; the scheduler runs on the submission node, and compute nodes never touch SQLite. The requirement is currently unassisted: `.oxymake/` is created in the working directory, and no flag, config key, or environment variable relocates it, so the state database cannot be separated from the workspace whose inputs and outputs are resolved against the same directory. An operator who needs the project tree on shared storage has no supported way to keep only the database local; a relocatable state directory is future work alongside multi-node coordination (multiple submission nodes), which requires a future coordination layer. This local-disk requirement is also the discharge of the `StateDbAtomicCommit` axiom that the model-checked multi-session guarantee rests on (§3.5).

Cache-key residual exclusions. Two ingredients are deliberately excluded from the cache key (Eq. 1). One is `call`-mode function bodies: the key covers the function *reference* (module path and name), but the module's source file is content-tracked only when declared as an input. Editing an imported module without redeclaring it can therefore serve a stale cached result; shell, inline-`run`, and script modes have no such gap (script file content is hashed into the key). The other is mutable container image tags: a `docker` or `apptainer` environment is hashed as the literal image reference, without resolving tags to digests—resolving would require a container-runtime round-trip per key computation and would break offline runs. A re-pushed `python:3.12-slim` therefore does not invalidate the cache; users who need this guarantee should pin images by digest (`python@sha256:...`), which the key then captures exactly.

In-memory passing limitations. The `call` mode's in-memory passing supports only types representable in the configured serialization format. Arbitrary Python objects (e.g., trained ML models) must be serialized to disk. A `pickle` codec provides a fallback but sacrifices cross-language compatibility.

Distributed executors. The local, Slurm, and Ray executors are fully implemented. The Kubernetes executor is designed (Section 4.3) but not yet implemented. The `Executor` trait and capability negotiation are in place; what remains is the backend-specific job submission, status polling, and log retrieval code for the Kubernetes backend.

Optimization passes. Of the six planned optimization passes (Section 4.1), only cache pruning is fully implemented. Task fusion, materialization elimination, group scheduling, critical path analysis, and partition planning are designed and have trait boundaries defined but await implementation. The current scheduler dispatches jobs in topological order without these optimizations; they will reduce execution time for `call`-mode workflows but do not affect correctness.

Evaluation statistics. The evaluation is limited in statistical depth as well as in scope. All measurements come from a single machine (Apple M4 Max, macOS, arm64); a Linux/x86-64 replication has not been performed, and no cross-platform claim is made. End-to-end times

are medians of three runs and resolution times are `hyperfine` medians; dispersion (IQR or confidence intervals) is not reported, and the 10,001-job end-to-end times are rounded to minutes scale. The reported ratios should be read with these limits in mind.

Maturity. OxyMake is in active development. The core engine (Table 5) implements the complete DAG pipeline from TOML parsing through parallel scheduling and execution (Section 6). The evaluated implementation is pre-1.0; its API may change.

7.4 Future Work: Substrate Composition

Future work will study whether OxyMake's orchestration-level cache contract composes with content-addressed package stores such as Guix [8, 12]. OxyMake's cache key records an *output-equivalence* relation (§3.1), while a content-addressed substrate records an *input-equivalence* relation: does this input hash produce this binary? The conjecture is that the two witnesses are orthogonal—each checkable independently, neither subsuming the other—so that their conjunction is a strictly stronger reproducibility statement than either alone, of the kind the guix-cwl reference workflows [31] illustrate at the substrate layer. This hypothesis is not evaluated here.

Rolling a substrate into the engine is explicitly *not* the intended path. Each layer of the reproducibility ladder (Table 1) has a distinct owner in the broader ecosystem—Guix-HPC for L1, build-systems literature for L3, the RO-Crate community [33] for L4—and an engine that collapses all four into one binary inherits all four maintenance burdens. A Guix-backed execution environment and a CWL reader/writer for `ox translate` (which today ships Snakemake and WDL bridges only) are therefore framed as invitations to a community-driven open-source effort rather than commitments of this paper.

7.5 A Note on Polyglot Layering

OxyMake assigns each layer a language chosen for the layer's constraints, not for uniformity. The orchestration core is Rust: the type system encodes pipeline-state invariants (`Rule` vs `ConcreteJob`) at compile time, and `Send + Sync` bounds make the concurrent scheduler thread-safe by construction. The workflow specification is TOML: deliberately not Turing-complete, statically parseable in microseconds, and analysable without execution—a property Snakemake's Python DSL cannot offer. Rule bodies execute in Python, R, Julia, or shell, selected per rule for the domain libraries each ecosystem encodes.

Each subsystem uses the language whose properties match its function, with serialisation contracts (structured IPC, file I/O) at the boundaries. The cost is an extra language frontier per layer; the benefit is that no single language has to compromise.

8 Conclusion

We have presented OxyMake, a workflow engine built on the position that the validity of a result is a property of the work rather than of the place it ran, and that this need not cost an operational system. The computation key is a function of declared content rather than a per-tree record: when the entire declared specification—rule source, inputs, parameters, environment, shell, platform—is unchanged, the key is identical in any checkout under the same key-format version. Identity is not reuse; the rebuild decision a checkout takes also consults its local key-to-output index, its materialised outputs, and the selected validation

policy (§3.1). The workflow is a declarative TOML file executed by a single 14.9 MB Rust binary with no daemon, and every command can emit structured NDJSON with `-json` for a human or a program. `ox run` ensures the declared outputs exist, runs only the jobs that are missing or out of date, and is safe to repeat.

The measured costs and benefits are bounded. On a synthetic 10,001-job workflow on one machine, DAG resolution takes 69 ms against Snakemake 7.32.4's 2.31 s; a cold end-to-end run is 1.25–2.3× slower, and a warm no-op re-run is 4.02–7.54× faster, depending on validation policy. Three TLA+ specifications model-check bounded safety of the concurrent state protocol. The cache key is computed from the declared specification only, so an undeclared read can cause stale reuse; output re-verification depth is a policy; and the claim protocol is not yet the scheduling gate. These limits are stated where they apply (§7.3).

OxyMake is open-source software under the Apache-2.0/MIT dual license, available at `https://github.com/noogram/oxymake` (`https://oxymake.noogram.dev`).

Acknowledgments

We thank the GNU Guix, CWL, and FAIR-workflows communities, whose prior work informed this engine's design.

We thank Michael R. Crusoe for comments on the descriptions of CWL and `cwltool`.

Disclosure of AI assistance

The development of OxyMake and the preparation of this manuscript made substantial use of large language model assistants, for code, prose drafting, and successive rounds of adversarial review. All design decisions, measurements, and final wording were verified by the author, who takes full responsibility for the content.

A TLA+ Specification Inventory

Table 15 summarizes the model-checking evidence behind §3.5: for each specification, the safety properties it checks, the bounds at which TLC explored the state space exhaustively, and the size of that exploration. The specifications, their TLC configuration files, and the archived TLC outputs are in the public repository under `spec/tla/`.

Module	Properties	Bounds	States	Depth
CacheConsistency.tla	OX-1, OX-6 (key determinism, stationary cache safety)	2 workers × 3 rules, worker crashes	7 436	16
CooperativeClaim.tla	INV-2 (claim atomicity, stale-session reclaim, zombie terminal writes)	3 sessions × 2 jobs, TTL 2, clock bound 4	1 663 056	23
CancelPropagation.tla	INV-3 (CancelledNeverCached, JobFailedImpliesNoIntent, NoZombieRunning)	3 jobs	32 768	22

Table 15: TLA+ specifications and their model-checking scope. States = distinct states explored by TLC at the stated bounds; Depth = BFS depth of the complete state graph. All checks assume the substrate axioms in the right column of Table 3 (SQLite commit atomicity on a local filesystem, executor honesty, atomic storage deletion, key purity, and input-declaration completeness). Two falsification configurations accompany the suite: each models a named defect (a terminal UPDATE without session filter; an undeclared input leaking into the cache key) and is expected to fail, which shows that the corresponding passing specification checks a property the defect can violate.

References

[1] Mainak Adhikari, Tarachand Amgoth, and Satish Narayana Srirama. A survey on scheduling strategies for workflows in cloud environment and emerging trends. *ACM Computing Surveys*, 52(4):1–36, 2019.

[2] Enis Afgan, Dannon Baker, Bérénice Batut, Marius van den Beek, Dave Bouvier, Martin Čech, John Chilton, Dave Clements, Nate Coraor, Björn A. Grüning, et al. The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update. *Nucleic Acids Research*, 46(W1):W537–W544, 2018.

[3] Argoproj contributors. Argo Workflows: The workflow engine for Kubernetes. `https://github.com/argoproj/argo-workflows`, 2018. CNCF graduated project. Accessed: 2026-04-01.

[4] Arvados Project. Arvados: An open source platform for managing and analyzing biomedical big data. `https://arvados.org/`, 2026. Open-source workflow and data platform combining a CWL runner with content-addressed data management (Keep) and provenance-tracked collections. Suggested by M. R. Crusoe (GitHub issue #1) as prior art for content-addressed workflow data.

[5] Bazel contributors. Starlark language specification. `https://github.com/bazelbuild/starlark/blob/master/spec.md`, 2018. Configuration language for Bazel, formerly Skylark. Accessed: 2026-04-01.

[6] Maxime Beauchemin and Apache Software Foundation. Apache Airflow: A platform to programmatically author, schedule, and monitor workflows. `https://airflow.apache.org`, 2015. Originally developed at Airbnb, Apache top-level project 2019. Accessed: 2026-04-01.

[7] Neil P. Chue Hong, Daniel S. Katz, Michelle Barker, Anna-Lena Lamprecht, Carlos Martinez, Fotis E. Psomopoulos, Jen Harrow, Leyla Jael Castro, Morane Gruenpeter, Paula Andrea Martinez, and Tom Honeyman. FAIR Principles for Research Software (FAIR4RS Principles). Recommendation, Research Data Alliance, 2022. Extends the FAIR Data Principles (Wilkinson 2016) to research software, adding software-specific indicators: machine-readable metadata, versioning, intelligible code, identifiable dependencies.

[8] Ludovic Courtès and Ricardo Wurmus. Reproducible and user-controlled software environments in HPC with Guix. In *Euro-Par 2015: Parallel Processing Workshops*, pages 579–591. Springer, 2015. GNU Guix as a from-source, content-addressed package manager for scientific computing. Establishes Guix's bit-reproducibility guarantee from input hash to binary output.

[9] Michael R. Crusoe, Sanne Abeln, Alexandru Iosup, Peter Amstutz, John Chilton, Nebojša Tijanić, Hervé Ménager, Stian Soiland-Reyes, Bogdan Gavrilović, Carole Goble, and The CWL Community. Methods included: Standardizing computational reuse and portability with the Common Workflow Language. *Communications of the ACM*, 65(6):54–63, 2022.

[10] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. Pegasus, a workflow management system for science automation. *Future Generation Computer Systems*, 46:17–35, 2015.

[11] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017.

[12] Eelco Dolstra. *The Purely Functional Software Deployment Model.* PhD thesis, Utrecht University, 2006.

[13] Stuart I. Feldman. Make – a program for maintaining computer programs. *Software: Practice and Experience*, 9(4):255–265, 1979.

[14] Carole Goble, Sarah Cohen-Boulakia, Stian Soiland-Reyes, Daniel Garijo, Yolanda Gil, Michael R. Crusoe, Kristian Peters, and Daniel Schober. FAIR Computational Workflows. *Data Intelligence*, 2(1-2):108–121, 2020.

[15] Gabriel Gonzalez. Dhall: A programmable configuration language, 2024. A total functional language that guarantees termination.

[16] Google. Bazel: A fast, scalable, multi-language and extensible build system. `https://bazel.build`, 2015. Open-source release of Google's internal Blaze build system. Accessed: 2026-04-01.

[17] Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012.

[18] Ruslan Kuprieiev et al. DVC: Data Version Control – Git for data & models. `https://dvc.org`, 2020.

[19] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.* Addison-Wesley, 2002.

[20] Meta Engineering. Build faster with Buck2: Our open source build system. `https://engineering.fb.com/2023/04/06/open-source/buck2-open-source-large-scale-build-system/`, 2023. Meta Engineering Blog. Accessed: 2026-04-01.

[21] Neil Mitchell. Shake before building: Replacing Make with Haskell. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming*, pages 55–66, 2012.

[22] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build Systems à la Carte. In *Proceedings of the ACM on Programming Languages*, volume 2, pages 1–29, 2018.

[23] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte: Theory and practice. *Journal of Functional Programming*, 30:e11, 2020.

[24] Felix Mölder, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twardziok, Alexander Kanitz, Andreas Wilm, Manuel Holtgrewe, Sven Rahmann, Sven Nahnsen, and Johannes Köster. Sustainable data analysis with Snakemake. *F1000Research*, 10:33, 2021.

[25] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX*

Symposium on Operating Systems Design and Implementation (OSDI 18), pages 561–577, 2018.

[26] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon Web Services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.

[27] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. `https://github.com/BLAKE3-team/BLAKE3`, 2020. Accessed: 2026-03-24.

[28] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3 specification. `https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf`, 2020. Benchmarks: 6.9 GiB/s single-threaded on Cascade Lake-SP 8275CL (16 KiB input); AVX-512 exceeds 8 GB/s. Accessed: 2026-04-01.

[29] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011.

[30] petgraph contributors. petgraph: Graph data structure library for Rust. `https://github.com/petgraph/petgraph`, 2024. $O(|V| + |E|)$ topological sort via Kahn's algorithm. Accessed: 2026-03-24.

[31] Pjotr Prins. CWL-workflows: Guix-managed Common Workflow Language reference workflows. Software repository, `https://github.com/pjotrp/CWL-workflows`, 2020. Software artifact (fork of hacchy1983/CWL-workflows). Reference workflows illustrating how a CWL runner composes with Guix-managed environments: orchestration (CWL/cwltool) and substrate (Guix store) are deliberately decoupled. README dated 2020. Accessed: 2026-05-28.

[32] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, pages 126–132, 2015.

[33] Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, José M. Fernández, Daniel Garijo, Björn Grüning, Lorraine La Rosa, Stefano Leo, Eoghan Ó Carragáin, Marc Portier, Ana Trisovic, RO-Crate Community, Paul Groth, and Carole Goble. Packaging research artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022. Specifies RO-Crate, a JSON-LD packaging format that bundles a workflow with its inputs, outputs, and provenance into a single FAIR-compliant artefact. Adopted by the WorkflowHub registry.

[34] Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. Enhancing reproducibility for computational methods. *Science*, 354(6317):1240–1241, 2016.

[35] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3):260–274, 2002.

[36] Marcel van Lohuizen. CUE: Validate, define, and use dynamic and text-based data, 2024. A constraint-based configuration language with types and validation.

[37] Kate Voss, Geraldine Van der Auwera, and Jeff Gentry. Full-stack genomics pipelining with GATK4 + WDL + Cromwell. F1000Research 6(ISCB Comm J):1381, 2017.

[38] Sean R. Wilkinson, Meznah Aloqalaa, Khalid Belhajjame, Michael R. Crusoe, Bruno de Paula Kinoshita, Luiz Gadelha, Daniel Garijo, Stian Soiland-Reyes, et al. Applying the FAIR Principles to computational workflows. *Scientific Data*, 12:328, 2025.

[39] Ricardo Wurmus, Bora Uyar, Brendan Osberg, Vedran Franke, Alexander Gosdschan, Katarzyna Wreçzycka, Jonathan Ronen, and Altuna Akalin. PiGx: Reproducible genomics analysis pipelines with GNU Guix. *GigaScience*, 7(12):giy123, 2018. Demonstrates Guix as the substrate layer for bioinformatics pipeline reproducibility; orchestration layer remains Snakemake-like.